

Corso di Laurea Magistrale in Ingegneria e Scienze Informatiche

Modernizzazione di una piattaforma ETL Spark-based tramite tecnologie Transactional Lakehouse su AWS

Relatore

Prof. Boschetti Marco Antonio

Candidato

Tommaso Brini

Correlatore

Dott. Mustafa Eduart (Prometeia S.p.A.)

Azienda

PROMETEIA S.P.A

Abstract

Questa tesi analizza la modernizzazione di una piattaforma ETL basata su Apache Spark e AWS Glue, precedentemente vincolata a un'architettura Data Lake tradizionale sul formato Parquet. L'obiettivo centrale del lavoro è l'introduzione del paradigma Transactional Lakehouse tramite l'adozione di Apache Iceberg. L'introduzione di questo table format ha permesso di abilitare proprietà ACID, funzionalità di time travel e una gestione avanzata dei metadati.

Nel corso del tirocinio è stato progettato e implementato un prototipo in grado di gestire pipeline complesse (INSERT, UPDATE, DELETE e MERGE) su dataset di grandi dimensioni, includendo una strategia automatizzata per la sincronizzazione del catalogo e dei metadati.

La soluzione proposta viene quindi confrontata con l'architettura esistente per valutarne gli effetti in termini di prestazioni, scalabilità e robustezza operativa, evidenziando vantaggi e limiti delle tecnologie lakehouse nel contesto di pipeline ETL in ambiente cloud.

Contents

Abstract	iii
1 Introduzione	1
2 Stato dell'arte e tecnologie	5
2.1 Evoluzione delle Architetture Dati	5
2.1.1 Il Data Warehouse Tradizionale	5
2.1.2 L'avvento del Data Lake	6
2.1.3 Paradigma Data Lakehouse	8
2.2 Open File Format e Apache Parquet	10
2.2.1 L'affermazione di Apache Parquet come standard	10
2.2.2 I limiti di un approccio basato solo su File Format	11
2.3 Apache Spark e l'elaborazione distribuita	13
2.4 Open Table Format e paradigma Lakehouse	14
2.4.1 Differenza tra File Format e Table Format	14
2.4.2 Il ruolo del Metadata Layer	15
2.4.3 Funzionalità Chiave dell'Architettura	15
2.5 Apache Iceberg: Architettura e funzionalità	17
2.5.1 Architettura dei metadata	17
2.5.2 Operazioni transazionali e Snapshot	20
2.5.3 Time Travel e Schema Evolution	22
2.5.4 Maintenance	23
2.6 Ecosistema AWS	23
3 Analisi As-is: Architettura Esistente	27
3.1 Pipeline giornaliera	27
3.1.1 Moduli della pipeline	30
3.2 Data Integration	31
3.2.1 Flow	31
3.2.2 Data	33
3.3 Configurazione delle Tabelle e dello storage	37

CONTENTS

3.3.1	Storage	39
3.4	Modalità di alimentazione: Full vs Delta	40
3.4.1	Modalità Full	40
3.4.2	Modalità Delta	41
3.4.3	Architettura Idempotente	42
3.4.4	Pro e Contro	44
3.5	Processo di Merge	44
3.6	Gestioni degli scarti	46
3.7	Ciclo di vita tra batch consecutivi	47
3.8	Flusso completo	49
3.9	Criticità dell'Architettura Corrente	49
3.9.1	Problema 1: Copia Fisica dei Dati tra Batch Consecutivi . .	50
3.9.2	Problema 2: Assenza di Idempotenza Nativa e di Meccanismi di Rollback	50
3.9.3	Problema 3: Aggiornamento Parziale su Record Compositi .	51
3.9.4	Problema 4: Portabilità tra Ambienti	52
3.9.5	Problema 5: Dipendenza dai Glue Crawler per la Sincronizzazione del Catalogo	52
3.9.6	Problema 6: Gestione del Compattamento dei File	53
3.10	Vincoli Trasversali	54
4	Progettazione e Implementazione	55
4.1	Requisiti progettuali	55
4.2	Scelta Tecnologica e Architetture	57
4.3	Progettazione della nuova pipeline ETL	58
4.3.1	Superamento del consolidamento fisico: eliminazione dello step FLOW	58
4.3.2	Gestione dello Stato e Idempotenza: il Cursor Watermark .	60
4.3.3	Evoluzione della Configurazione	62
4.4	Amazon S3 Tables e Automazione della Maintenance	63
4.5	Migrazione dei metadata	63
4.6	Gestione dei record compositi	63
5	Test e risultati	65
6	Conclusioni e sviluppi futuri	67
	Bibliography	69

Chapter 1

Introduzione

Il presente lavoro di tesi è stato sviluppato nell’ambito di un tirocinio svolto presso Prometeia S.p.A., azienda attiva nella consulenza e nello sviluppo di soluzioni software per il settore finanziario.

Negli ultimi anni la crescita esponenziale dei dati generati da applicazioni distribuite, sistemi transazionali e servizi cloud ha reso necessaria l’evoluzione delle architetture aziendali per la gestione e l’analisi dei dati. In questo contesto, i processi di **Extract, Transform, Load (ETL)** hanno assunto un ruolo centrale nelle pipeline di data engineering, richiedendo soluzioni sempre più scalabili, affidabili e flessibili.

La diffusione dei paradigmi Big Data ha favorito l’adozione dei Data Lake, architetture basate su storage distribuito in grado di memorizzare e gestire grandi quantità di dati eterogenei a costi ridotti. Formati colonnari come Apache Parquet hanno quindi rappresentato uno standard de facto per l’elaborazione distribuita tramite framework come Apache Spark. Nel modello Lakehouse, i Data Lake moderni si basano infatti su storage cloud economico e formati aperti per supportare workload analitici e di machine learning. [AGX⁺21] [BGK21]

Nonostante i vantaggi in termini di costo e scalabilità, l’approccio tradizionale basato su file Parquet presenta alcune limitazioni significative. In particolare, i Data Lake classici non forniscono nativamente proprietà transazionali ACID, una

gestione evoluta dei metadati o un supporto efficiente per operazioni di aggiornamento incrementale. Operazioni come UPDATE, DELETE e MERGE richiedono quindi meccanismi applicativi complessi, spesso basati sulla riscrittura di intere partizioni o dataset, con un aumento della complessità delle pipeline ETL e del rischio di inconsistenze nei dati.

Per superare tali criticità si è diffuso il paradigma del Data Lakehouse, che combina la flessibilità dei Data Lake con caratteristiche tipiche dei Data Warehouse, introducendo transazioni, versionamento dei dati, schema evolution e una gestione avanzata dei metadati. [Sah24]

Tra le tecnologie emergenti in questo ambito, Apache Iceberg si è affermato come uno dei principali table format open-source per la gestione di dataset analitici su larga scala. Iceberg introduce un layer di metadata management che separa la struttura logica della tabella dalla disposizione fisica dei file su storage object-based, consentendo la gestione di snapshot consistenti, time travel e operazioni incrementali transazionali. [Apa25a]

A differenza di un approccio puramente file-based, Iceberg mantiene lo stato della tabella attraverso metadata versionati e manifest file, consentendo aggiornamenti atomici mediante commit espliciti. [Apa25b]

Questa tesi si inserisce nel contesto della modernizzazione di una piattaforma ETL aziendale basata su Apache Spark e servizi AWS. L'architettura esistente utilizza file Parquet memorizzati su Amazon S3 ed elaborati tramite AWS Glue, presentando criticità legate alla gestione degli aggiornamenti incrementali, alla sincronizzazione del catalogo e alla garanzia di idempotenza dei processi batch. Queste limitazioni si traducono operativamente in inefficienze specifiche, formalizzate dall'azienda in alcuni punti critici, che compromettono la robustezza della pipeline e la coerenza del dato nel tempo.

L'obiettivo del lavoro è progettare e valutare una nuova architettura basata sul paradigma Transactional Lakehouse attraverso l'introduzione di Apache Iceberg. In particolare, il lavoro analizza l'integrazione del table format con Apache Spark e l'ecosistema AWS, includendo aspetti relativi alla gestione dei metadati, all'esecuzione di operazioni MERGE transazionali e ai processi di maintenance delle tabelle. Durante il progetto sono inoltre state esplorate le funzionalità offerte da **Amazon S3 Tables** [Ama25a], soluzione managed introdotta da AWS per la

gestione di tabelle Iceberg.

Il contributo principale della tesi consiste nella progettazione e implementazione di un prototipo in grado di eseguire operazioni incrementali affidabili e idempotenti su dataset di grandi dimensioni, confrontando la nuova soluzione con l'architettura preesistente in termini di prestazioni, scalabilità e robustezza. L'analisi sperimentale evidenzia inoltre i trade-off introdotti dall'adozione del paradigma lakehouse, valutando vantaggi e limiti sia di un'implementazione Iceberg tradizionale sia dell'approccio gestito offerto da S3 Tables.

Nei prossimi capitoli, la tesi introdurrà lo stato dell'arte relativo alle architetture Data Lake e Lakehouse, descrivendo le principali tecnologie utilizzate (Capitolo 2). Successivamente, verrà presentata un'analisi dettagliata dell'architettura esistente e delle criticità riscontrate (Capitolo 3), seguita dalla progettazione e implementazione della nuova soluzione basata su Apache Iceberg (Capitolo 4). Infine, i risultati sperimentali e i benchmark effettuati saranno discussi nel Capitolo 5, raccogliendo conclusioni e possibili sviluppi futuri nel Capitolo 6.

Chapter 2

Stato dell'arte e tecnologie

2.1 Evoluzione delle Architetture Dati

L'evoluzione delle architetture dati è strettamente legata alla crescita del volume, della varietà e della velocità con cui le informazioni vengono generate e processate all'interno dei sistemi informativi moderni.

La necessità di estrarre valore da sorgenti dati eterogenee ha guidato l'evoluzione architetturale dai tradizionali database relazionali fino ai moderni ecosistemi distribuiti.

Nel corso degli anni, le piattaforme analitiche hanno attraversato diverse fasi evolutive, passando dai sistemi relazionali tradizionali ai Data Warehouse, successivamente ai Data Lake e, più recentemente, al paradigma Lakehouse.

2.1.1 Il Data Warehouse Tradizionale

Nel contesto dei tradizionali sistemi di gestione di database relazionali (RDBMS), le piattaforme sono state storicamente suddivise in sistemi per l'*Online Transactional Processing* (OLTP) e sistemi per l'*Online Analytical Processing* (OLAP).

I primi sistemi OLAP per la gestione dati per finalità analitiche erano basati su architetture di tipo **Data Warehouse**. Tali sistemi si fondano su schemi relazionali fortemente strutturati e su processi ETL volti a trasformare e normalizzare i dati prima del caricamento.

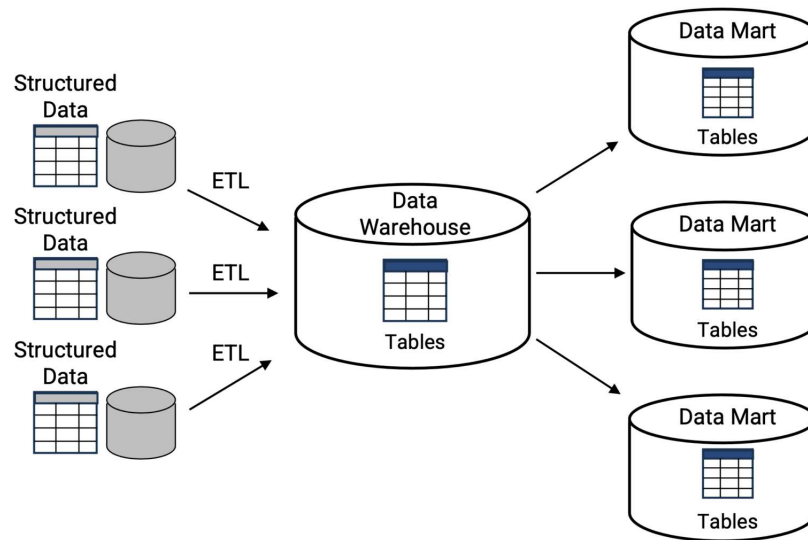


Figure 2.1: Data Warehouse Architecture. [Nix24]

Questo approccio garantisce elevata affidabilità, proprietà ACID e buone performance per reporting e business intelligence. Inoltre, questi sistemi garantiscono ottime prestazioni di interrogazione grazie all'utilizzo di partizionamenti e indicizzazioni avanzate.

Tuttavia, l'imposizione di formati di dati strettamente strutturati ne limita fortemente la flessibilità rispetto alla gestione di dati eterogenei e workload avanzati di analytics e machine learning. I costi di archiviazione su queste piattaforme sono molto elevati, l'infrastruttura risulta rigida di fronte alla necessità di scalare computazionalmente e l'architettura chiusa porta frequentemente al fenomeno del *vendor lock-in*.

2.1.2 L'avvento del Data Lake

Per superare l'incapacità dei Data Warehouse di gestire dati non strutturati e semi-strutturati e con l'ulteriore aumento delle quantità di dati prodotti da applicazioni web, dispositivi mobili e sistemi distribuiti, parallelamente alla diffusione delle tecnologie Big Data è emerso il concetto di **Data Lake**.

Questa architettura sfrutta file system distribuiti come *Hadoop HDFS* e, più recentemente, sistemi di archiviazione a oggetti in cloud a basso costo (come Amazon S3), permettendo di memorizzare enormi quantità di dati grezzi a costi estremamente contenuti. In questo contesto, l'introduzione di formati di file aperti e colonnari, come Apache Parquet e ORC, ha portato notevoli benefici in termini di interoperabilità e prestazioni di interrogazione all'interno del Data Lake.

La differenza principale rispetto al Data Warehouse è quindi la possibilità di memorizzare dati strutturati, semi-strutturati e non strutturati senza la necessità di un processo di trasformazione preventiva, garantendo una maggiore flessibilità e scalabilità.

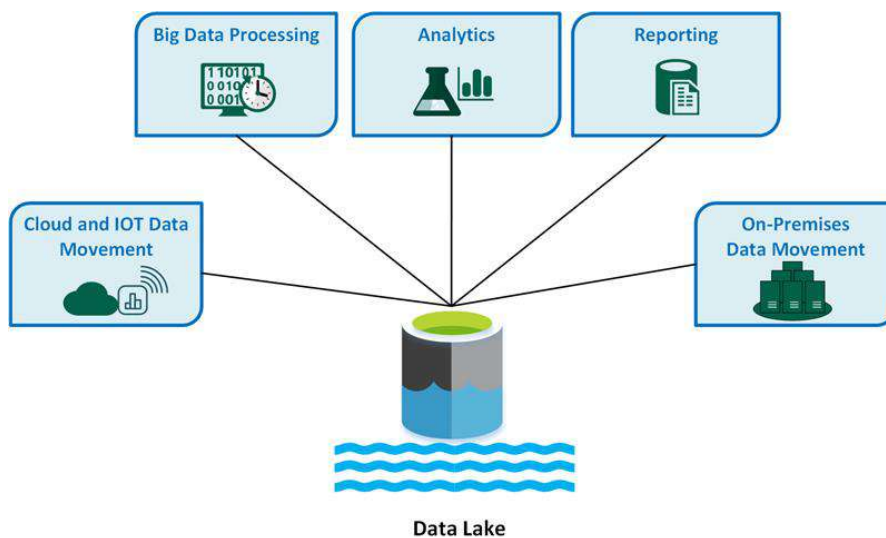


Figure 2.2: Data Lake Architecture. [el]

Nonostante l'enorme flessibilità e i vantaggi in termini di costo e capacità di ingestione, i Data Lake tradizionali hanno mostrato importanti limitazioni operative, che spesso li trasformano in ecosistemi difficilmente gestibili, noti come **Data Swamp** (letteralmente, paludi di dati).

In particolare, l'incapacità di imporre uno schema rigido e l'impossibilità di garantire transazioni ACID espongono il sistema a frequenti rischi di corruzione dei dati in caso di fallimenti dei job di scrittura. Inoltre, l'assenza di funzionalità

transazionali, la gestione limitata dei metadati e la difficoltà nel supportare operazioni incrementali hanno spesso portato alla creazione di pipeline ETL complesse e difficili da mantenere. Inoltre, molte organizzazioni hanno adottato architetture ibride in cui i dati venivano continuamente replicati tra Data Lake e Data Warehouse, aumentando costi, latenza e complessità gestionale.

2.1.3 Paradigma Data Lakehouse

Il Data Lakehouse è il concetto architettonico nato per superare i limiti intrinseci di Data Warehouse e Data Lake, offrendo una soluzione dati unificata. Questa nuova architettura riesce a fondere i due mondi, portando la conformità ACID e l'affidabilità dei dati tipiche dei Data Warehouse sopra l'archiviazione a basso costo, scalabile e interoperabile dei Data Lake.

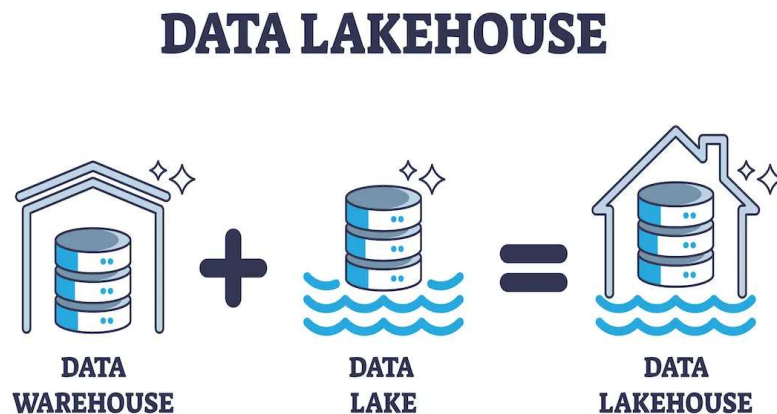


Figure 2.3: Data Lakehouse Architecture. [Ach24]

Un Lakehouse è una piattaforma basata su formati aperti e storage object-based che fornisce supporto nativo per analytics, machine learning e gestione transazionale dei dati. [AGX⁺21]

Le architetture Lakehouse introducono infatti caratteristiche quali:

- proprietà ACID;
- gestione evoluta dei metadati;
- versionamento e time travel;
- supporto a operazioni UPDATE, DELETE e MERGE;
- interoperabilità tra differenti motori di elaborazione.

Tali funzionalità vengono generalmente implementate tramite Open Table Format, come **Apache Iceberg**, Delta Lake e Apache Hudi, che aggiungono un layer logico di gestione sopra i file presenti nel Data Lake.

Questo approccio consente di mantenere la flessibilità e il basso costo dello storage object-based, introducendo al contempo meccanismi di consistenza e affidabilità tipici dei sistemi warehouse tradizionali.

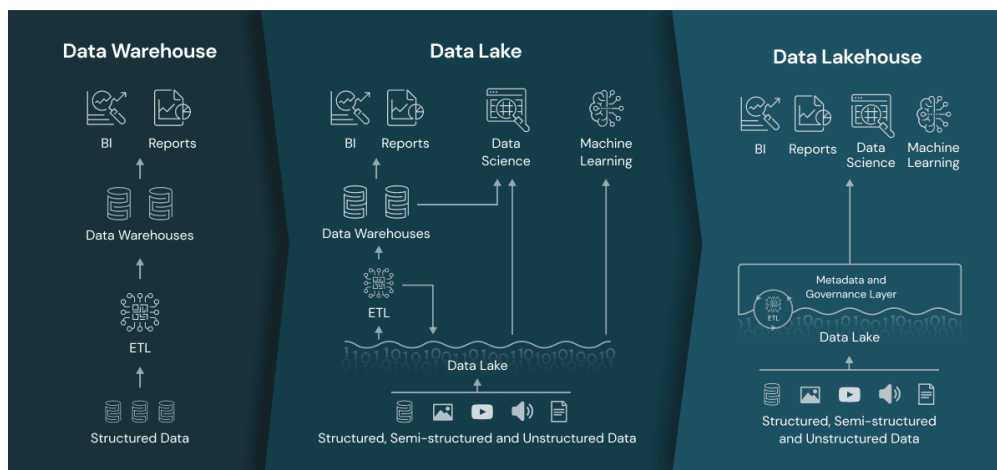


Figure 2.4: Data Architectures. [Sta24]

2.2 Open File Format e Apache Parquet

La diffusione delle architetture Big Data e dei Data Lake ha favorito l'adozione di *Open File Format* progettati per supportare l'elaborazione distribuita di grandi volumi di dati. L'obiettivo principale di questi open format è garantire l'interoperabilità tra diversi motori di calcolo e piattaforme cloud, evitare costose duplicazioni dei dati, ridurre i costi di archiviazione e ottimizzare le prestazioni di accesso alle informazioni.

All'interno di questo panorama, i formati si dividono principalmente in due categorie: orientati per riga (Row-oriented) e orientati per colonna (Column-oriented). Mentre i formati per riga (come Apache Avro o i tradizionali CSV) sono ideali per i sistemi transazionali tipici dei DBMS, i formati colonnari sono stati introdotti nei sistemi Big Data per massimizzare le performance analitiche.

2.2.1 L'affermazione di Apache Parquet come standard

Tra i formati maggiormente diffusi in ambito analitico vi è Apache Parquet, un formato colonnare open-source sviluppato per ottimizzare le performance di lettura e compressione nei workload analitici distribuiti. Parquet è stato progettato per integrarsi con ecosistemi Big Data come Apache Hadoop e Apache Spark, diventando progressivamente uno standard de facto per la memorizzazione di dataset nei Data Lake. [Apa25c]

A differenza dei tradizionali formati row-based, nei quali i dati vengono memorizzati riga per riga, Parquet utilizza un'organizzazione column-oriented. In un formato row-based, i valori appartenenti alla stessa riga vengono salvati consecutivamente sul disco, rendendo efficiente l'accesso transazionale ai record completi. Tuttavia, nei workload analitici *OLAP* è spesso necessario leggere solo un sottoinsieme delle colonne disponibili; in tali scenari, l'approccio columnar consente di ridurre significativamente il volume di dati letto da storage e migliorare le performance delle query.

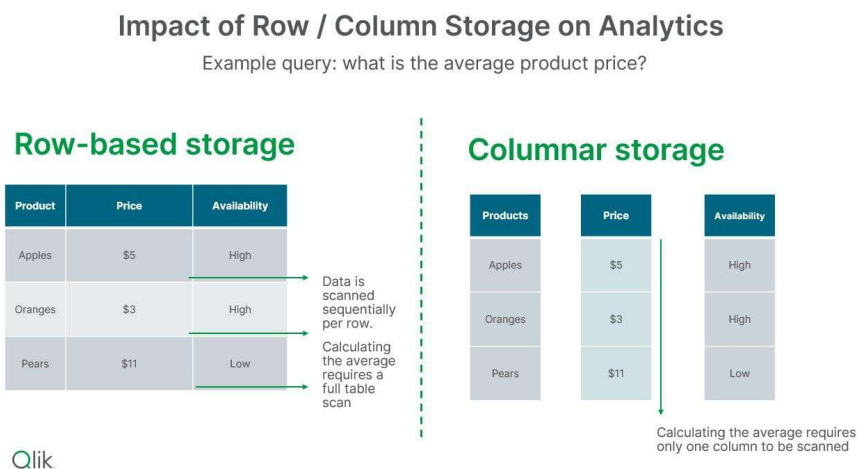


Figure 2.5: Row-oriented vs Column-oriented storage. [Sta25]

L'organizzazione colonnare offre inoltre vantaggi rilevanti in termini di compressione dei dati. Poiché i valori appartenenti alla stessa colonna tendono ad avere struttura e distribuzione simili, algoritmi di encoding e compressione risultano più efficaci rispetto ai formati row-based. Questo permette di ridurre i costi di storage e migliorare l'efficienza delle operazioni di scansione.

Inoltre, un ulteriore elemento che ha favorito l'adozione di Parquet è la sua natura open-source e il supporto per un ampio ecosistema di strumenti e piattaforme cloud, garantendo interoperabilità e flessibilità nella gestione dei dati. Infatti, il formato è supportato nativamente da numerosi framework e query engine, tra cui **Apache Spark**, **Amazon Athena**, **Trino**. Tale caratteristica ha reso possibile la costruzione di Data Lake aperti e indipendenti dal motore di elaborazione utilizzato.

2.2.2 I limiti di un approccio basato solo su File Format

Nonostante questi vantaggi, l'utilizzo di semplici file Parquet presenta alcune limitazioni significative nei moderni workload Lakehouse. Parquet è infatti un file

format e non un table format: esso definisce esclusivamente il modo in cui i dati vengono memorizzati all'interno dei file, senza fornire funzionalità avanzate di gestione transazionale o metadata management.

Nello specifico, un Data Lake basato esclusivamente su Parquet non supporta nativamente:

- **proprietà ACID**
- **operazioni UPDATE e DELETE efficienti**
- **gestione transazionale concorrente**
- **versionamento dei dati e time travel**
- **schema evolution avanzata**

Queste limitazioni hanno portato alla nascita degli **Open Table Format**, tecnologie progettate per estendere le capacità dei file format tradizionali introducendo gestione avanzata dei metadati, snapshot transazionali e supporto a operazioni incrementali. Tra le principali soluzioni emerse in questo ambito vi sono Apache Iceberg, Delta Lake e Apache Hudi, che rappresentano il fondamento delle moderne architetture Lakehouse.

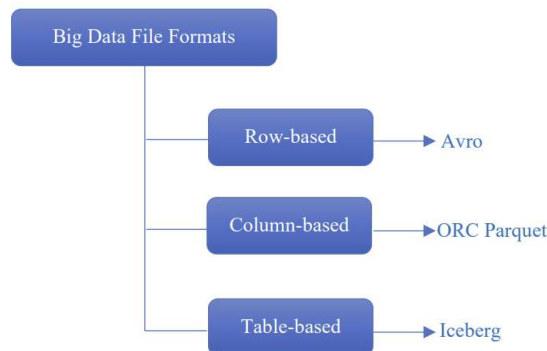


Figure 2.6: Different File Formats [NS25]

2.3 Apache Spark e l'elaborazione distribuita

La crescente diffusione dei Big Data richiede motori computazionali in grado di scalare orizzontalmente in modo efficiente. In questo contesto, Apache Spark rappresenta uno dei principali framework utilizzati nelle moderne architetture Data Lake e Lakehouse grazie alle sue capacità di parallelizzazione, scalabilità e integrazione con differenti sistemi di storage. [Apa25d]

Apache Spark è un framework open-source per il calcolo distribuito, progettato per garantire elevate prestazioni attraverso l'elaborazione in-memory. Rispetto ai precedenti paradigmi, come il classico modello MapReduce, Spark riduce drasticamente le operazioni di lettura e scrittura su disco, risultando particolarmente adatto a workload analitici e pipeline ETL complesse.

L'architettura di Spark si basa su un modello master-worker. Il processo centrale, chiamato Driver, ospita la logica applicativa e coordina l'esecuzione distribuendo i task ai vari Executor eseguiti sui nodi Worker del cluster. Questi ultimi elaborano i dati in parallelo sfruttandone la suddivisione in partizioni distribuite, massimizzando l'efficienza computazionale.

L'interazione con il framework avviene tipicamente attraverso API ad alto livello, in particolare DataFrame e Spark SQL. Tali interfacce consentono di manipolare dataset distribuiti in forma tabellare mediante un approccio dichiarativo, simile a quello dei database relazionali tradizionali.

Grazie alla sua natura distribuita, Spark può integrarsi con differenti sistemi di storage e formati dati, tra cui Apache Parquet, Apache Iceberg e object storage cloud come Amazon S3. Questa interoperabilità ha favorito la sua diffusione come motore principale per processi ETL e workload analitici in ambienti cloud-native.

Spark si occupa esclusivamente di elaborare i dati e non fornisce nativamente proprietà transazionali ACID. Per garantire atomicità, consistenza e affidabilità nelle operazioni distribuite, è quindi necessario integrare il motore computazionale con un layer logico avanzato di gestione delle tabelle, rappresentato dagli Open Table Format.

2.4 Open Table Format e paradigma Lakehouse

Il paradigma Lakehouse trova la sua effettiva realizzabilità tecnica nell'introduzione di un livello software intermedio noto come *Open Table Format*. Questo strato logico si pone l'obiettivo di colmare il divario tra la flessibilità dell'archiviazione di file su object storage e la rigorosa gestione tabellare tipica dei sistemi relazionali tradizionali.

2.4.1 Differenza tra File Format e Table Format

L'evoluzione delle moderne architetture Data Lake ha evidenziato i limiti dei tradizionali file format nella gestione di workload analitici sempre più complessi. Sebbene formati colonnari come Apache Parquet offrano elevate prestazioni, essi definiscono esclusivamente la rappresentazione fisica dei dati all'interno dei file senza introdurre meccanismi avanzati di gestione transazionale o controllo dello stato delle tabelle. Di conseguenza, nelle architetture tradizionali basate solo su file format, i motori di interrogazione dovevano scansionare intere directory del file system distributed o dell'object storage per determinare quali file appartenessero a una determinata tabella, causando inefficienze prestazionali e seri problemi di consistenza in caso di modifiche concorrenti.

In questo contesto emergono gli Open Table Format, che risolvono alla radice tale limite spostando la definizione di "verità" della tabella dal percorso fisico della directory a una gestione centralizzata basata sui metadati.

Infatti, tali tecnologie introducono un layer logico di gestione sopra i file presenti nello storage distribuito. A differenza quindi dei semplici file format, un table format non si limita a descrivere come i dati vengono memorizzati, ma definisce anche:

- la struttura logica della tabella;
- la gestione dei metadati;
- le operazioni transazionali;
- il versionamento dei dati.

Gli Open Table Format rappresentano il fondamento tecnologico del paradigma Lakehouse, architettura nata per combinare la flessibilità e la scalabilità dei Data Lake con le funzionalità tipiche dei Data Warehouse tradizionali. [AGX⁺21]

2.4.2 Il ruolo del Metadata Layer

Il nucleo tecnologico di un Open Table Format è il *metadata layer*. Questo strato è costituito da un insieme strutturato di file che tracciano e documentano lo stato e la composizione dei file fisici presenti nello storage (in formato Parquet, ORC o Avro).

Quando un motore di calcolo interagisce con il sistema, non dialoga direttamente con i file di dati grezzi, ma interroga il metadata layer. Quest'ultimo astrae la complessità dei file fisici e fornisce una vista tabellare coerente, ottimizzando l'accesso e garantendo l'integrità del dato.

2.4.3 Funzionalità Chiave dell'Architettura

Gli Open Table Format introducono una struttura *metadata-driven* in cui ogni modifica alla tabella genera una nuova versione consistente dello stato dei dati. Questo modello consente di implementare proprietà ACID anche in ambienti distribuiti basati su object storage, garantendo atomicità e isolamento delle operazioni.

Tra le principali funzionalità abilitate dagli Open Table Format vi sono:

- operazioni CRUD transazionali
- supporto a operazioni MERGE incrementali
- snapshot isolation
- partition evolution
- time travel e query storiche

In particolare, il time travel permette di accedere a versioni precedenti della tabella attraverso snapshot consistenti.

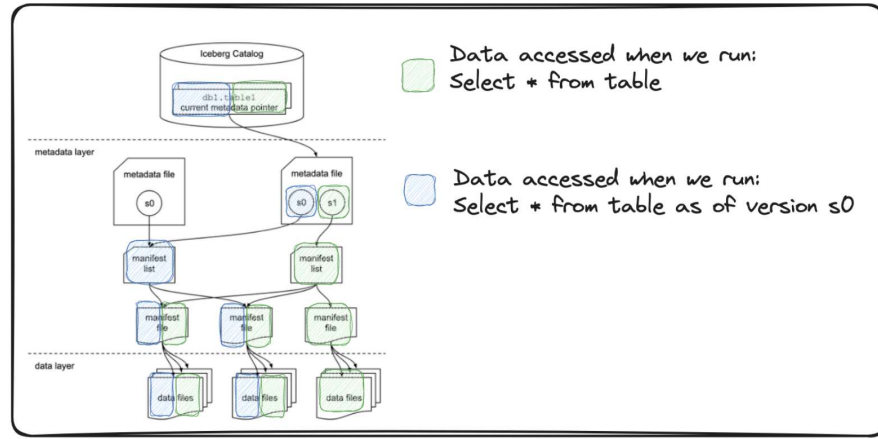


Figure 2.7: Time Travel in Open Table Formats. [Mac23]

La schema evolution consente invece di modificare la struttura logica delle tabelle senza richiedere la riscrittura completa dei dataset, migliorando la flessibilità delle pipeline ETL.

Le principali implementazioni Open Table Format diffuse nel panorama Lakehouse sono Apache Iceberg, Delta Lake e Apache Hudi. Sebbene condividano l'obiettivo di introdurre funzionalità transazionali nei Data Lake, tali tecnologie adottano approcci differenti nella gestione dei metadati, delle operazioni incrementali e della compatibilità con i diversi motori di elaborazione.

Nel contesto di questa tesi, l'attenzione è rivolta principalmente ad Apache Iceberg.

2.5 Apache Iceberg: Architettura e funzionalità

Apache Iceberg è uno degli Open Table Formats più diffusi, progettato per portare il comportamento, la flessibilità e l'affidabilità tipici del linguaggio SQL all'interno degli ecosistemi Big Data. Ideato originariamente da Netflix per superare i limiti operativi di Apache Hive e successivamente donato alla Apache Software Foundation, Iceberg introduce un layer avanzato di metadata management che consente di implementare transazioni ACID, versionamento dei dati e operazioni incrementali affidabili su object storage cloud-based. [Sah24]

Le immagini e le tabelle di questo capitolo sono state prese dal tutorial di Iceberg realizzato da Confluent Developer [DB26].

2.5.1 Architettura dei metadata

L'infrastruttura di Iceberg si discosta dagli altri table format per la sua complessa architettura dei metadata di natura gerarchica e stratificata. Iceberg introduce infatti un'architettura *metadata-driven* che separa la rappresentazione logica della tabella dalla disposizione fisica dei file nello storage distribuito, consentendo di mantenere snapshot consistenti della tabella e di tracciare tutte le modifiche effettuate nel tempo.

L'architettura di Iceberg si basa su diversi livelli logici, organizzati gerarchicamente:

- **Catalog:** rappresenta il punto di accesso principale alle tabelle Iceberg e contiene il riferimento al metadata file corrente della tabella. Iceberg supporta differenti tipologie di catalogo, tra cui Hive Metastore, REST Catalog e AWS Glue Catalog.

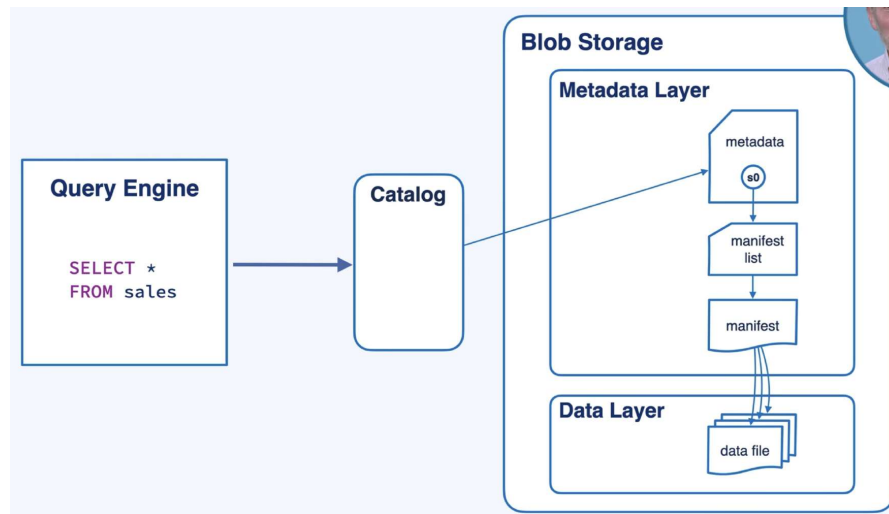


Figure 2.8: Apache Iceberg Catalog Architecture. [DB26]

- **Metadata File:** è il livello centrale dell'architettura. Si tratta di un file in formato JSON che contiene lo stato completo della tabella, i suoi snapshot storici, lo schema e le specifiche di partizionamento.
- **Manifest List:** è un file in formato Avro che contiene un elenco di file manifest e mantiene i valori limite (inferiore e superiore) delle colonne di partizionamento per facilitare il filtraggio durante le query. Ogni snapshot della tabella punta infatti a una manifest list differente, consentendo di ricostruire lo stato esatto della tabella in un determinato momento temporale.
- **Manifest File:** è il livello più basso dei metadati (in formato Avro) che traccia l'elenco puntuale dei file di dati fisici (come Parquet) in cui risiedono effettivamente le informazioni di una data partizione.
- **Data File:** i Data File rappresentano i file fisici contenenti i dati veri e propri, generalmente memorizzati in formato Apache Parquet.

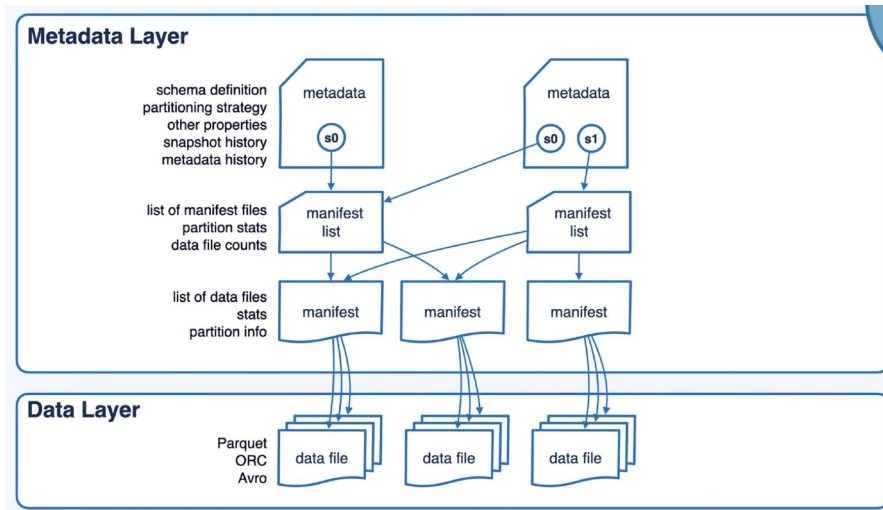


Figure 2.9: Apache Iceberg Metadata Architecture. [DB26]

Questa struttura gerarchica consente a Iceberg di separare completamente il layout fisico dei dati dalla rappresentazione logica della tabella. A differenza delle tradizionali tabelle Hive, nelle quali la struttura delle partizioni dipende direttamente dall'organizzazione delle directory sul filesystem, Iceberg mantiene tutte le informazioni di mapping nei metadati, introducendo maggiore flessibilità e semplificando l'evoluzione della struttura delle tabelle [Sah24].

Nel contesto delle pipeline ETL distribuite, questo modello consente di implementare processi più robusti e idempotenti, riducendo la necessità di logiche applicative dedicate alla sincronizzazione dei file e alla gestione manuale dello stato dei dataset. Infatti, ogni modifica alla tabella genera un nuovo metadata file, rendendo il commit dell'operazione atomico e versionato. Questo approccio consente di mantenere la consistenza della tabella senza modificare direttamente i file dati esistenti.

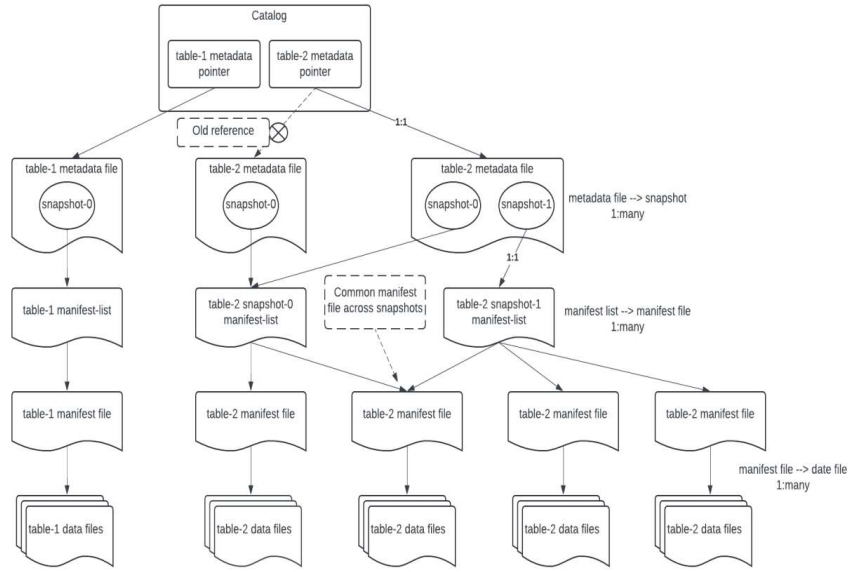


Figure 2.10: Table's Metadata File Structure. [Sah24]

2.5.2 Operazioni transazionali e Snapshot

Nei Data Lake tradizionali basati su semplici file Parquet, le modifiche ai dataset vengono generalmente effettuate tramite riscrittura di file o partizioni, senza alcuna garanzia nativa di atomicità e consistenza. Questo approccio può introdurre inconsistenze in caso di scritture concorrenti, errori durante l'esecuzione dei job ETL o riesecuzioni parziali delle pipeline.

Iceberg garantisce l'atomicità delle operazioni (ACID) definendo il concetto di snapshot, ovvero una vista coerente, catturata in un preciso istante, dell'intera tabella inclusi dati e metadati.

Lo **snapshot** rappresenta quindi una vista completa e immutabile del dataset in uno specifico istante temporale.

Il sistema permette letture e scritture simultanee attraverso un approccio di Optimistic Concurrency Control (OCC). In questo modello, più processi possono operare contemporaneamente sugli stessi dataset senza utilizzare lock espliciti sui file dati. Al termine dell'operazione, il sistema verifica che il metadata file corrente non sia stato modificato da altri processi concorrenti. In caso contrario, il commit

viene rifiutato e l'operazione deve essere rieseguita.

Dal punto di vista operativo, ogni commit in Iceberg produce nuovi data file, nuovi manifest file, una nuova manifest list e un nuovo metadata file. Il commit diventa effettivo solamente quando il catalogo aggiorna il riferimento al nuovo metadata file, rendendo atomica la transizione tra lo stato precedente e quello aggiornato della tabella.

Una delle principali conseguenze di questo modello è il supporto nativo a operazioni incrementali avanzate, come INSERT, UPDATE, DELETE e MERGE. In particolare, l'operazione MERGE INTO rappresenta uno degli elementi centrali nelle moderne pipeline ETL incrementali, consentendo di combinare dati nuovi e dati esistenti in maniera transazionale.

Le operazioni di aggiornamento introdotte da Iceberg possono essere implementate attraverso differenti modalità di scrittura, progettate per bilanciare efficienza delle letture e costo delle operazioni incrementali.

In particolare, Apache Iceberg supporta due approcci principali: **Copy-on-Write (CoW)** e **Merge-on-Read (MoR)**.

Nel modello Copy-on-Write, le modifiche ai dati comportano la riscrittura completa dei file coinvolti nell'operazione. Quando viene eseguita un'azione di aggiornamento, Iceberg genera nuovi data file contenenti la versione aggiornata dei record e aggiorna successivamente i metadata della tabella tramite un nuovo snapshot. Questo approccio garantisce elevate performance in lettura e una struttura dei dati relativamente semplice, poiché le query accedono direttamente ai file aggiornati senza necessità di elaborazioni aggiuntive.

Nel modello Merge-on-Read, invece, le modifiche vengono registrate tramite file incrementali separati, come delete file o update file, che vengono successivamente combinati in fase di lettura. Questo approccio riduce il costo delle scritture incrementali e risulta particolarmente adatto a workload caratterizzati da aggiornamenti frequenti o streaming continuo. Tuttavia, introduce maggiore complessità nelle operazioni di lettura e rende necessarie attività periodiche di compaction per consolidare i file generati. È la scelta preferenziale per sistemi soggetti a piccole e frequenti scritture.

	Copy-on-Write (CoW)	Merge-on-Read (MoR)
Write Performance	✗ Slow - rewrites entire file	✓ Fast - writes delete files + changes
Read Performance	✓ Fast - direct file reads	✗ Slow - merges delta at read time
Storage Overhead	✗ High - duplicates data on updates	✓ Low - stores only changes
Table Maintenance	✓ Low - files already optimized	✗ High - needs frequent compaction
Best For	Read-heavy workloads Analytics (BI, reports)	Write-heavy workloads (Streaming, CDC)

Figure 2.11: Copy-on-Write vs Merge-on-Read. [DB26]

2.5.3 Time Travel e Schema Evolution

Il sofisticato livello dei metadati abilita una serie di funzionalità distintive cruciali per le pipeline ETL:

- **Time Travel:** Essendo basato su snapshot immutabili, ogni operazione di modifica genera una nuova versione senza sovrascrivere fisicamente i vecchi dati immediatamente. Questo permette agli utenti di "viaggiare nel tempo" interrogando lo stato storico dei dati semplicemente specificando un timestamp o l'ID di uno specifico snapshot senza dover duplicare fisicamente i dataset. Tale approccio risulta particolarmente utile in scenari di auditing, debugging delle pipeline ETL, recupero da errori applicativi o analisi storica delle modifiche.
- **Schema Evolution:** In Iceberg, le modifiche allo schema (aggiunta, rimozione o ridenominazione di colonne) interessano esclusivamente i file di metadati. I file Parquet sottostanti rimangono inalterati, permettendo al sistema di evolvere senza il rischio di corrompere l'architettura.

- **Partition Evolution:** Iceberg gestisce la logica di partizionamento a livello di metadati tramite una tecnica definita *hidden partitioning*. Ciò consente di evolvere in modo dinamico gli schemi di partizionamento qualora le prestazioni degradino, senza richiedere agli utenti finali o ai sistemi ETL di alterare la sintassi delle loro query.

2.5.4 Maintenance

L'introduzione di un layer avanzato di metadata management e di funzionalità transazionali rende necessario implementare specifiche attività di maintenance per garantire efficienza e stabilità nel tempo delle tabelle Apache Iceberg. A differenza dei tradizionali Data Lake basati esclusivamente su file Parquet, infatti, Iceberg mantiene molteplici versioni dei metadata e snapshot storici, generando progressivamente nuovi file ad ogni commit.

Per risolvere questa problematica, l'architettura richiede processi di manutenzione periodica. In particolare, la gestione dei file di dati e dei metadata prevede attività di compaction e pulizia che consentono di consolidare i file frammentati e rimuovere i vecchi snapshot obsoleti.

Il processo di **compaction** si occupa di unire in background i file di piccole dimensioni in file più capienti, oltre a riconciliare fisicamente i delete file con i dati originali.

Tra le operazioni di pulizia implementate da Iceberg, troviamo come lo **snapshot expiration** e la gestione degli **orphan files**. La prima si occupa di eliminare definitivamente gli snapshot obsoleti e i relativi file di dati e metadata, liberando spazio e riducendo la complessità, mentre la seconda si occupa di identificare e rimuovere i file di dati non più referenziati da alcuno snapshot, evitando l'accumulo di file storici, mitigando i costi infrastrutturali di storage.

2.6 Ecosistema AWS

La diffusione delle architetture Lakehouse è stata favorita anche dall'evoluzione dei servizi cloud, che consentono di realizzare pipeline ETL distribuite sfruttando componenti scalabili e completamente gestiti. In questo contesto, Amazon Web

Services (AWS) offre un ecosistema particolarmente adatto alla costruzione di piattaforme analitiche basate su Apache Spark e Open Table Format. [Ama25b]

Amazon S3 come Data Lake Storage

Uno degli elementi centrali dell'architettura è Amazon Simple Storage Service (S3), servizio di *object storage* utilizzato per memorizzare dataset, metadata e file Parquet. Grazie alla sua elevata scalabilità, durabilità e integrazione con i principali motori di elaborazione distribuita, S3 rappresenta uno degli storage più diffusi per implementazioni Data Lake e Lakehouse cloud-native.

AWS Glue: Catalog e Jobs

Per l'elaborazione distribuita dei dati, AWS mette a disposizione **AWS Glue**, servizio serverless basato su Apache Spark progettato per l'esecuzione di processi ETL. Glue consente di sviluppare job Spark senza la necessità di gestire direttamente infrastrutture cluster dedicate, integrandosi nativamente con servizi AWS come S3, Glue Catalog e Athena.

Per coordinare l'accesso ai dati, il sistema utilizza AWS Glue Data Catalog. In un'architettura Iceberg, questo servizio assume il ruolo fondamentale di *Catalog* di livello superiore. Funge da metastore centralizzato, memorizzando i riferimenti e i puntatori all'ultimo file di metadati valido per ogni tabella, garantendo così l'integrità e l'atomicità delle operazioni concorrenti.

Amazon Athena

Amazon Athena è un servizio di interrogazione interattiva serverless che permette di analizzare i dati memorizzati su S3 utilizzando la sintassi SQL standard. Grazie all'integrazione nativa con Apache Iceberg, Athena consente di eseguire query *ad-hoc* e operazioni di *Time Travel* direttamente sulle tabelle del Lakehouse, beneficiando dell'ottimizzazione prestazionale garantita dai metadati.

Amazon S3 Tables

Recentemente, AWS ha introdotto Amazon S3 Tables, una soluzione managed progettata specificamente per la gestione di tabelle Apache Iceberg su Amazon S3. A differenza dell'implementazione Iceberg tradizionale appoggiata su AWS Glue Catalog, nella quale le onerose operazioni di maintenance devono essere orchestrate manualmente tramite job Spark dedicati, S3 Tables demanda l'ottimizzazione direttamente al livello di storage.

La piattaforma automatizza in modo nativo attività critiche come la compaction dei file, la snapshot expiration e la rimozione dei file orfani.

S3 Tables introduce inoltre concetti come namespace e table bucket, semplificando l'organizzazione logica delle tabelle Iceberg e la loro integrazione con i servizi AWS.

Nel contesto di questa tesi, l'ecosistema AWS rappresenta la piattaforma tecnologica utilizzata per progettare e validare la soluzione proposta, integrando Apache Spark, Iceberg e object storage cloud-based all'interno di una pipeline ETL.

Chapter 3

Analisi As-is: Architettura Esistente

L'analisi della situazione attuale deriva da uno studio approfondito dell'architettura della libreria e dalla documentazione tecnica fornita dall'azienda. Il seguente capitolo è quindi frutto di questa analisi e si concentra sull'identificazione dei principali *pain points* e delle inefficienze riscontrate nella pipeline ETL esistente. Tutte le tabelle e le immagini di questo capitolo sono state prese dalla documentazione aziendale.

3.1 Pipeline giornaliera

L'azienda offre una libreria per integrare e elaborare flussi di dati provenienti dalle varie banche e clienti.

Questa libreria è progettata come un tool per integrare ed elaborare flussi di dati, tipicamente ottenuti sotto forma di file testuali, nell'ambito della realizzazione di un Personal Financial Planner (PFP).

Utilizza tecnologie come Apache Spark e Scala, e si appoggia sui servizi cloud offerti dall'infrastruttura AWS, consentendo agli sviluppatori di evitare di riscrivere procedure appartenenti a pattern largamente diffusi e utilizzati in Prometeia. Offre anche un alto grado di affidabilità e scalabilità rispetto alla quantità e numerosità dei dati processati.

La libreria si compone di diversi moduli che possono essere integrati e modificati a seconda delle esigenze del progetto. I moduli principali sono:

- **Integrazione flussi:** punto di ingresso di una qualsiasi Data Integration PFP Cloud. Nasce con l'obiettivo di processare e validare flussi di dati provenienti da sistemi alimentanti esterni (tipicamente il cliente) e da sorgenti dati interne
- **Catalogo prodotti:** fornisce una procedura per la creazione del catalogo prodotti, utilizzato sia dal PFP che dai moduli batch successivi
- **Clienti:** Fornisce una procedura per l'aggregazione delle informazioni appartenenti al singolo cliente. Il risultato viene utilizzato sia dal PFP che dagli eventuali moduli batch successivi
- **Rendimenti:** in questo modulo vengono calcolati massivamente i rendimenti periodici dei clienti appartenenti al perimetro di calcolo

Il progetto template che utilizza la libreria è infatti strutturato nella seguente maniera:

```
{tenant}-scala-spark/
├─ src/main/scala/it/{company}/
│  ├─ MainData.scala           # Entry point principale
│  ├─ flow/                    # Logica di smistamento file
│  ├─ etl/                     # Logica di data integration
│  ├─ catalog/                 # Logica catalogo prodotti
│  ├─ customer/               # Logica clienti
│  └─ returns/                 # Logica rendimenti
├─ src/main/resources/
│  └─ conf/tables/             # Configurazioni JSON tabelle
│     ├─ input/                # Tabelle di input
│     └─ output/               # Mapping di output
├─ .run/
│  └─ MainData.run.xml         # Run configuration IntelliJ
├─ DAS/                        # File statici
└─ pom.xml
```

Figure 3.1: Struttura del progetto template.

3.1. PIPELINE GIORNALIERA

Il Main principale (MainData) gestisce il routing verso i runner dei vari moduli, in base al *-runnerMode*.

```
object MainData {
  def main(sysArgs: Array[String]): Unit = {
    val config = new EnvironmentConfig(sysArgs)

    config.runnerMode match {
      case Some("FLOW") => MainFlow.run(config)
      case Some("DATA") => MainETL.run(config)
      case Some("PRODUCT_CATALOG") => MainProductCatalog.run(config)
      case Some("CUSTOMERS_GENERATION") => MainCustomersGenerator.run(config)
      case Some("CUSTOMERS_WRITE") => MainCustomersWriter.run(config)
      case Some("RETURNS") => MainReturns.run(config)
      case None => MainETL.run(config)
      case _ => throw new RuntimeException("Invalid runner mode")
    }
  }
}
```

Figure 3.2: Main Data.

Il sistema adotta un pattern di **dynamic dispatch** basato sulla configurazione dell'ambiente: il parametro **RunnerMode**, passato a runtime, determina quale implementazione del runner viene istanziata tramite pattern matching, disaccoppiando la logica di orchestrazione dall'implementazione specifica di ciascun modulo.

Questi moduli vengono orchestrati in una pipeline giornaliera che si occupa di elaborare i flussi di dati provenienti dalle banche, trasformarli in un formato analitico e renderli disponibili per le applicazioni frontend di analisi e reporting.

La pipeline giornaliera viene orchestrata da AWS Step Functions che esegue sequenzialmente AWS Glue Jobs, corrispondenti ai vari moduli della libreria.

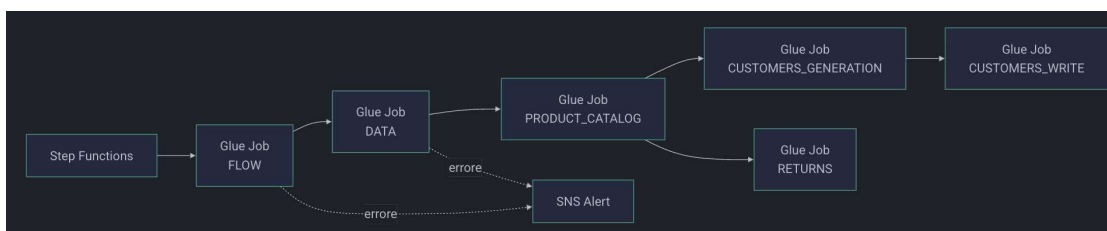


Figure 3.3: Step Function.

3.1.1 Moduli della pipeline

Come detto, la pipeline giornaliera si compone di diversi moduli eseguiti sequenzialmente, ognuno dei quali si occupa di una specifica fase del processo ETL.

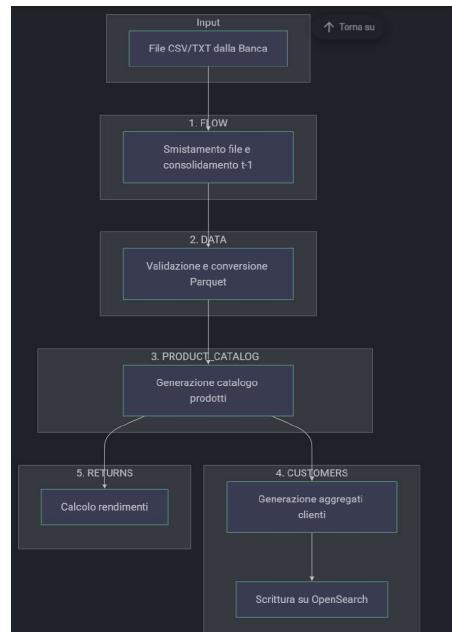


Figure 3.4: Pipeline giornaliera.

In questo progetto, l'attenzione è rivolta principalmente al modulo di integrazione dei flussi, che rappresenta il punto di partenza della pipeline e si occupa di validare e convertire i flussi csv in Parquet.

3.2 Data Integration

Il modulo di Data Integration è il punto di ingresso della pipeline dati. È progettato per processare e validare flussi di dati eterogenei provenienti da sistemi alimentanti esterni e interni, indipendentemente dal volume dei dati trattati.

Il processo di ingestione si articola in due step fondamentali:

- **FLOW**: Smistamento dei file ricevuti e consolidamento dello stato del giorno precedente ($t-1$)
- **DATA**: Validazione, trasformazione e conversione dei flussi in formato Parquet, con eventuale merge con lo storico.

L'output di questo modulo consiste in:

- **File Parquet validati**, utilizzabili dai moduli successivi della pipeline.
- File Parquet contenenti i **record scartati**, corredati di metadati sul motivo dello scarto.

3.2.1 Flow

Lo step FLOW è il primo della pipeline giornaliera. Prepara l'ambiente di lavoro per l'elaborazione del giorno corrente (T) a partire dai risultati del giorno precedente (T-1).

Operazioni

1. **Smistamento dei file della banca**: i file depositati nella cartella input/ vengono distribuiti nelle sottocartelle import/{TABLE-NAME}/, ciascuna corrispondente a una tabella configurata.
2. **Copia file statici**: i file di configurazione e mappatura presenti in DAS/ vengono copiati in import/

3. **Consolidamento t-1:** lo stato prodotto dal batch del giorno precedente viene copiato nelle cartelle di input per il giorno corrente:

- output/store/ → input/store/
- output/errors/ → input/errors/

Lo storico del giorno T-1 diventa l'input del giorno T. Gli scarti possono essere recuperati se corretti nel flusso successivo.

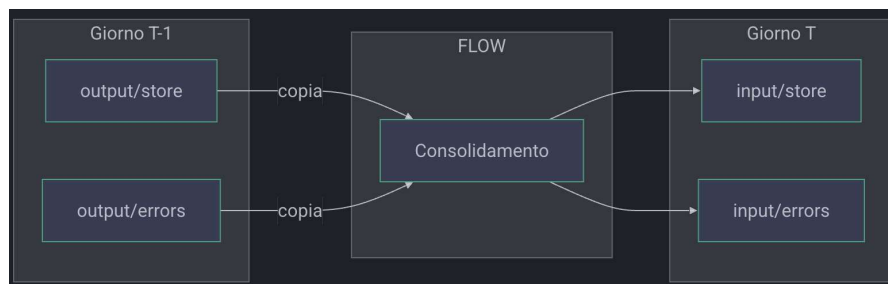


Figure 3.5: Consolidamento dello stato t-1 nello step FLOW

Idempotenza

La separazione tra percorsi di input e output garantisce una forma di idempotenza *condizionale*: rieseguire lo stesso batch prima del consolidamento produce lo stesso risultato.

Tuttavia, una volta consolidati i dati del giorno, una riesecuzione accidentale di FLOW sovrascriverebbe lo stato corrente in modo irreversibile. Questo rivela un limite intrinseco della gestione a file nudi su S3: in assenza di transazionalità, il sistema non dispone di meccanismi nativi per distinguere una prima esecuzione da una riesecuzione tardiva. La responsabilità di prevenire tale scenario ricade interamente sull'orchestratore (AWS Step Functions), che deve implementare logiche esplicite di controllo dello stato prima di avviare ogni esecuzione, aumentando la complessità operativa della pipeline.

Struttura

Dopo l'esecuzione dello step FLOW, la struttura delle cartelle sul bucket è la seguente:

```
bucket/  
  import/                                # File smistati (pronti per DATA)  
    ANAGR_CLIENTI/  
    SALDI/  
    MOVIMENTI/  
  input/                                  # Storico consolidato da t-1  
    store/  
      ANAGR_CLIENTI/  
      MOVIMENTI/  
    errors/  
      ANAGR_CLIENTI/  
      MOVIMENTI/  
  DAS/                                    # File statici (mappature, config)
```

Figure 3.6: Struttura delle cartelle dopo lo step FLOW

3.2.2 Data

Lo step DATA è il cuore del processo di ingestione, eseguito a seguito del completamento dello step FLOW.

In questo step vengono caricati i file smistati da FLOW, successivamente vengono validati e trasformati, e infine vengono convertiti in formato Parquet.

Come detto, l'output di questo modulo consiste in:

- Un insieme di file, tipicamente in formato parquet, validati, che possono essere utilizzati come sorgente dati dai moduli successivi.
- un insieme di file, in formato parquet, che contengono i record ritenuti anomali a seconda della configurazione utilizzata.

Workflow

Ogni tabella configurata attraverso 7 fasi sequenziali.

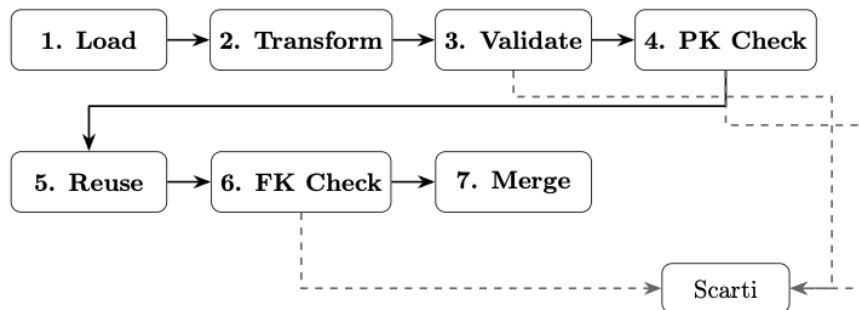


Figure 3.7: Le 7 fasi del workflow

Al fine di risolvere conflitti di dipendenze circolari, è stato aggiunta un ultimo step successivo al merge, denominato *Post Execution*. In questa fase vengono eseguite operazioni di post-processing e le trasformazioni finali con accesso a tutte le tabelle processate.

L'ordine effettivo di esecuzione delle fasi è quindi il seguente:

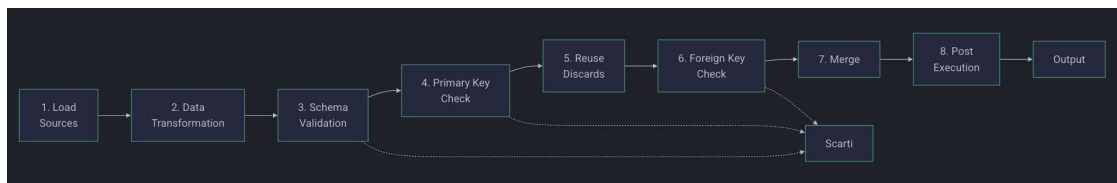


Figure 3.8: Workflow completo della pipeline DATA

Tra le fasi è possibile inserire trasformazioni personalizzate chiamate **Hook** per modificare i dati.

Gli *hook* sono inseribili nel workflow attraverso codice personalizzato inserito tramite il trait Transformations.

3.2. DATA INTEGRATION

I punti di aggancio disponibili sono:

Hook	Momento di Esecuzione
BeforeDataValidation	Prima della validazione dei dati
AfterColumnsTransformation	Dopo la trasformazione delle colonne
AfterColumnsValidation	Dopo la validazione schema
AfterDuplicatesRemoval	Dopo la rimozione dei duplicati
AfterDiscardedRecordsReuse	Dopo il recupero degli scarti
AfterFkConstraintsCheck	Dopo il controllo delle FK
AfterMerge	Dopo il merge con lo storico
AfterDataValidation	Dopo tutte le validazioni

Table 3.1: Hook di trasformazione disponibili

Il workflow data si compone quindi delle sopracitate fasi alternate dall'esecuzione degli hook personalizzati, che consentono di modificare i dati in qualsiasi punto del processo.

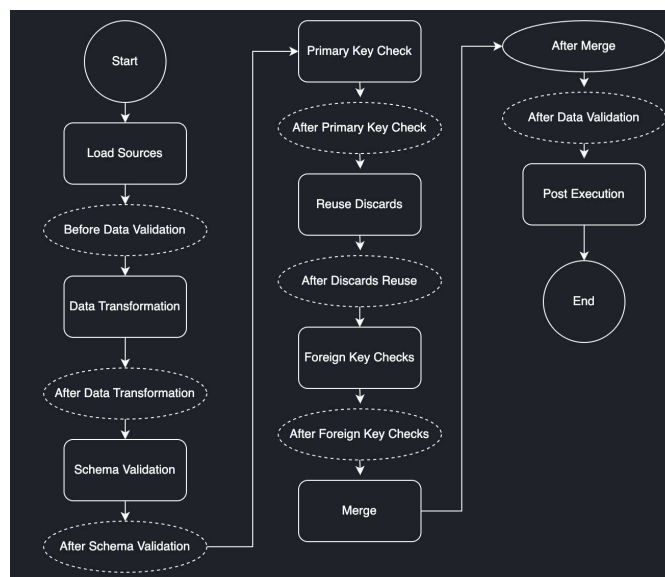


Figure 3.9: Workflow completo del caricamento dati

3.2. DATA INTEGRATION

La tabella seguente riassume le 7 fasi del workflow DATA, evidenziando le operazioni principali eseguite in ciascuna fase e il loro scopo all'interno del processo di ingestione dei dati. Verranno anche assegnati i punti di aggancio degli hook personalizzati, che consentono di inserire logiche di trasformazione specifiche in momenti chiave del workflow.

#	Fase	Descrizione	Hook
1	Load Sources	Caricamento file da S3 o filesystem locale (CSV, Parquet, Positional)	<i>beforeDataValidation</i>
2	Data Transformation	Applica trasformazioni alle colonne (fill, constant, pad, take, expr, map) + mascheramento dati sensibili	<i>afterColumnsTransformation</i>
3	Schema Validation	Cast automatico ai tipi definiti, controllo campi obbligatori (nullable: false), vincoli di dominio. Record non validi → scarti	<i>afterColumnsValidation</i>
4	Primary Key Check	Rimozione duplicati su primaryKey . Duplicati → scarti con motivo DUPLICATE	<i>afterDuplicatesRemoval</i>
5	Reuse Discards	Recupero scarti riutilizzabili da iErrors (record con REUSE=S). Left anti join con dati correnti	<i>afterDiscardedRecordsReuse</i>
6	Foreign Key Check	Verifica integrità referenziale con tabelle padre. Record senza match → scarti con FK.VIOLATION	<i>afterFkConstraintsCheck</i>
7	Merge	Merge con storico da iStorage (modalità delta). Gestione updateOption e record perpetui	<i>afterMerge,</i> <i>afterDataValidation</i>
8	Post-Execution	Trasformazioni finali con accesso a tutte le tabelle processate. Usato per risolvere dipendenze circolari	<i>postExecution</i>

Table 3.2: Le 8 fasi del pipeline DATA

3.3 Configurazione delle Tabelle e dello storage

Per la configurazione delle tabelle è stata consultata la documentazione tecnica fornita dall'azienda, che descrive in dettaglio la struttura dei file di configurazione e le convenzioni adottate per l'organizzazione dei dati su S3.

Ogni tabella è definita da un file JSON che regola il suo intero ciclo di vita nell'ingestione. Il JSON di configurazione consente di definire in modo flessibile e centralizzato tutte le caratteristiche della tabella, tra cui:

- Identificazione e formato (name, format, bucket, ecc..)
- Percorsi sorgente (source, multipleSources, ecc..)
- Percorsi di output (oStorage, oErrors, oDeltaStorage)
- Percorsi di input per storico e scarti (iStorage, iErrors)
- Schema dei dati (colonne, tipi, ecc..)
- Vincoli e dipendenze (chiavi primarie, chiavi esterne, dipendenze tra tabelle)
- Opzioni aggiuntive (colonne per partizionamento, forzare caching del DataFrame su disco, configurazione export, ecc..)

3.3. CONFIGURAZIONE DELLE TABELLE E DELLO STORAGE

I campi principali sono riassunti nella seguente tabella.

Campo	Tipo	Descrizione
name	String	Nome univoco della tabella
format	Enum	Formato sorgente: CSV, PARQUET, POSITIONAL
source	String	Percorso del file sorgente (supporta wildcard)
iStorage	String	Storico precedente per merge (modalità delta)
oStorage	String	Output dati validati
iErrors	String	Scarti precedenti per riutilizzo
oErrors	String	Output scarti
oDeltaStorage	String	Output solo record delta
columns	List	Definizione dello schema (nome, tipo, nullable)
primaryKey	List[String]	Colonne della chiave primaria
foreignKey	List	Vincoli di integrità referenziale
dependencyList	List[String]	Tabelle da elaborare prima

Table 3.3: Campi principali della configurazione tabella

Per esempio, la tabella `movimenti` è configurata dal seguente file JSON.

```
{
  "name": "MOVIMENTI",
  "format": "CSV",
  "header": true,
  "sep": ";",
  "source": "import/MOVIMENTI/movimenti.*",
  "iStorage": "input/store/MOVIMENTI",
  "oStorage": "output/full/MOVIMENTI",
  "iErrors": "input/errors/MOVIMENTI",
  "oErrors": "output/errors/MOVIMENTI",
  "oDeltaStorage": "output/delta/MOVIMENTI",
  "columns": [
    {"name": "ID_MOVIMENTO", "datatype": "String", "nullable": false},
    {"name": "CODICE_CLIENTE", "datatype": "String", "nullable": false},
    {"name": "IMPORTO", "datatype": "Double"}
  ],
  "primaryKey": ["ID_MOVIMENTO"],
  "foreignKey": [
    {"parent": "CLIENTI", "columns": ["CODICE_CLIENTE"]}
  ]
}
```

Figure 3.10: Esempio di configurazione tabella.

3.3.1 Storage

I dati vengono memorizzati su Amazon S3, organizzati in cartelle che seguono una convenzione basata sui percorsi definiti nella configurazione di ciascuna tabella.

Di norma, dopo l'esecuzione dello step DATA il bucket S3 presenta la struttura mostrata nella seguente figura.

```
bucket/  
  import/                                # File smistati da FLOW (input per DATA)  
    ANAGR_CLIENTI/  
    SALDI/  
    MOVIMENTI/  
  input/                                  # Storico consolidato (t-1, copiato da  
    FLOW)  
  store/  
    ANAGR_CLIENTI/  
    MOVIMENTI/  
  errors/  
    ANAGR_CLIENTI/  
    MOVIMENTI/  
  output/  
    full/                                 # Output completo (storico + delta)  
      ANAGR_CLIENTI/  
      SALDI/  
      MOVIMENTI/  
    delta/                                # Solo record nuovi/modificati  
      ANAGR_CLIENTI/  
      MOVIMENTI/  
  errors/                                  # Scarti  
    ANAGR_CLIENTI/  
    MOVIMENTI/  
  DAS/                                    # File statici (mappature, configurazioni  
  )
```

Figure 3.11: Struttura del bucket S3 dopo lo step DATA.

Tutti i dati validati e gli scarti sono salvati in formato Apache Parquet, che garantisce compressione efficiente, schema tipizzato e lettura colonnare ottimizzata per Spark.

3.4 Modalità di alimentazione: Full vs Delta

La scelta tra modalità Full e Delta dipende da come l'alimentante invia i dati.

Il sistema implementa due modalità di alimentazione distinte, progettate per adattarsi alle diverse caratteristiche dei flussi di dati in ingresso e alle esigenze di aggiornamento dei dataset.

- **Modalità Full:** l'alimentante invia ogni giorno l'intero dataset.
- **Modalità Delta:** l'alimentante invia ogni giorno solo i record nuovi o modificati.

3.4.1 Modalità Full

L'alimentante invia ogni giorno l'intero dataset. In questa modalità, non è necessario alcun merge con lo storico: ogni esecuzione produce un output completo e indipendente.

Caratteristiche

- Nessun merge con storico
- Non richiede iStorage
- L'output (oStorage) viene sovrascritto ad ogni esecuzione, ogni giorno.
- Più semplice da gestire, adatta a volumi piccoli/medi

Configurazione

Listing 3.1: Esempio di configurazione in modalità Full

```
1 {  
2   "name": "CLIENTI",  
3   "source": "input/CLIENTI.csv",  
4   "oStorage": "output/CLIENTI",  
5   "oErrors": "errors/CLIENTI",  
6   "primaryKey": ["CODICE_CLIENTE"]  
7 }
```

Percorsi Coinvolti

Percorso	Tipo	Descrizione
source	Input	File giornaliero completo
oStorage	Output	Dati validati (sovrascrive ogni giorno)
oErrors	Output	Scarti del giorno

Table 3.4: Percorsi coinvolti nella modalità Full

3.4.2 Modalità Delta

L'alimentante invia ogni giorno solo i record nuovi o modificati.

E' la modalità più delicata da gestire, in quanto richiede il merge con lo storico del giorno precedente e una gestione accurata degli scarti.

Caratteristiche

- Merge con storico t-1
- Richiede iStorage e oStorage separati
- Supporta riutilizzo scarti
- Ottimale per grandi volumi con poche modifiche giornaliere
- $iStorage \neq oStorage$ e $iErrors \neq oErrors$

Configurazione

Listing 3.2: Esempio di configurazione in modalità Delta

```

1 {
2   "name": "MOVIMENTI",
3   "source": "import/MOVIMENTI/movimenti.csv",
4   "iStorage": "input/store/MOVIMENTI",
5   "oStorage": "output/full/MOVIMENTI",
6   "iErrors": "input/errors/MOVIMENTI",
7   "oErrors": "output/errors/MOVIMENTI",
8   "oDeltaStorage": "output/delta/MOVIMENTI",
9   "primaryKey": ["CODICEBANCA", "CODICE"]
10 }
```

Percorsi Coinvolti

Percorso	Tipo	Descrizione
source	Input	File giornaliero (solo nuovi/modificati)
iStorage	Input	Storico precedente (t-1)
iErrors	Input	Scarti precedenti (per riutilizzo)
oStorage	Output	Risultato completo (storico + nuovi)
oErrors	Output	Scarti aggiornati
oDeltaStorage	Output	Solo record nuovi/modificati del giorno

Table 3.5: Percorsi coinvolti nella modalità Delta

3.4.3 Architettura Idempotente

In modalità Delta, l'idempotenza non può essere garantita nativamente dal merge di file Parquet: riscrivere lo stesso dataset su percorsi condivisi tra esecuzioni consecutive renderebbe indistinguibile uno stato corretto da uno corrotto.

Il meccanismo adottato si basa sulla **separazione esplicita tra percorsi di input e percorsi di output**: ogni batch legge dallo stato prodotto dal batch precedente (iStorage, iErrors) e scrive su percorsi distinti (oStorage, oErrors).

Al termine dell'esecuzione, un processo notturno (step FLOW) promuove gli output del giorno a input per il batch successivo, avanzando lo stato in modo controllato.

Listing 3.3: Ciclo di vita tra batch consecutivi in modalità Delta

```
1 Batch Day T:
2   Read:    iStorage (t-1) + source (delta)
3   Process: validazione, merge
4   Write:   oStorage (t0), oErrors (t0)
5
6 AWS Copy (nightly):
7   oStorage (t0) -> iStorage (t0)
8   oErrors (t0) -> iErrors (t0)
9
10 Batch Day T+1:
11   Read:    iStorage (t0) + source (delta)
12   Process: validazione, merge
13   Write:   oStorage (t1), oErrors (t1)
```

Questo schema offre le seguenti garanzie:

- **Idempotenza:** rieseguire il batch del giorno T produce sempre lo stesso `oStorage (t0)`, poiché gli input `iStorage (t-1)` e `source` non vengono modificati dall'esecuzione.
- **Tracciabilità:** input e output di ogni giorno rimangono distinti e ispezionabili.
- **Rollback:** in caso di errore è sufficiente non promuovere gli output, mantenendo lo stato precedente come input valido per una riesecuzione.
- **Testabilità:** è possibile rieseguire batch su ambienti di test senza alterare i dati di produzione.

3.4.4 Pro e Contro

Criterio	Full	Delta
Alimentante invia	Tutto ogni giorno	Solo modifiche
Volume dati	Piccolo/medio	Grande
Modifiche giornaliere	Molte	Poche
Storicizzazione	Non necessaria	Necessaria
Riutilizzo scarti	Non supportato	Supportato
Complessità	Bassa	Media

Table 3.6: Confronto tra modalità Full e Delta

3.5 Processo di Merge

Per la modalità Delta, la fase di merge rappresenta il principale step critico della pipeline. Come descritto in precedenza, rappresenta l'ultimo step del workflow DATA, precedendo solamente la fase di *Post Execution*.

Si occupa di combinare i dati validati del giorno corrente con lo storico del giorno precedente, producendo un output completo e aggiornato. Opera sulla base della **primaryKey** configurata nel JSON di configurazione per ogni tabella.

Logica di Merge

Dati due insiemi:

- il **batch corrente**, cioè i record nuovi o modificati del giorno corrente, letti da **source**
- lo **storico**, cioè i dati validati al giorno t-1, letti da **iStorage**

il merge produce il dataset completo secondo queste regole, basate sulla chiave primaria:

1. **Record presenti solo nello storico** (chiave non presente nel batch corrente): vengono *mantenuti* nel risultato finale

2. **Record presenti solo nel batch corrente** (chiave non presente nello storico): vengono *inseriti* nel risultato finale
3. **Record presenti in entrambi** (stessa chiave primaria): il record del batch corrente *sovrascrive* quello storico

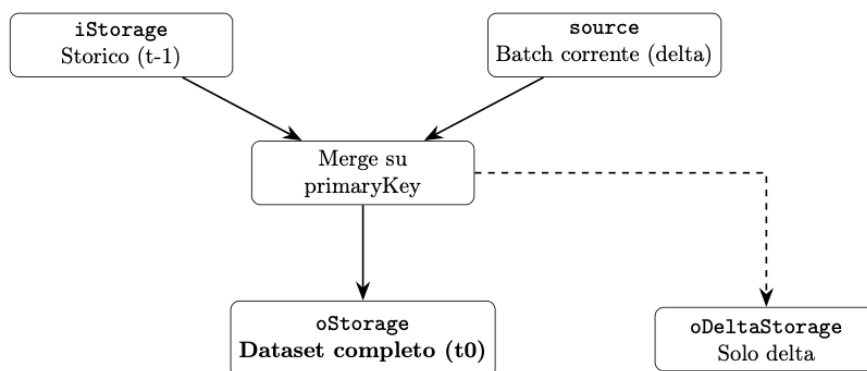


Figure 3.12: Logica di merge tra storico e batch corrente.

Opzioni di aggiornamento per colonna

Nel file JSON di configurazione è possibile configurare il comportamento di merge a livello di singola colonna tramite il campo `updateOption`.

Modalità	Comportamento
ALWAYS	Il valore del nuovo record sovrascrive sempre lo storico.
NEVER	Il valore storico viene sempre mantenuto; il nuovo viene ignorato.
ON_VALUE	Il nuovo valore sovrascrive lo storico solo se diverso da <code>ignoredValue</code> (es. stringa vuota).

Table 3.7: Modalità di aggiornamento colonna durante il merge

Un'altra opzione possibile, sempre configurabile a livello di configurazione, è marcare un record come perpetuo. Infatti, i record marcati con `isPerpetual = true` vengono sempre mantenuti dallo storico, anche se presenti nel batch corrente. Questa funzionalità è utile per dati che non devono mai essere sovrascritti dall'alimentante.

3.6 Gestioni degli scarti

Gli scarti rappresentano i record che per qualche ragione non superano le fasi di validazione e vengono quindi esclusi dal risultato finale. Questi record vengono quindi salvati in file Parquet separati arricchiti di colonne di metadati.

Le colonne aggiuntive di metadati sono le seguenti:

Colonna	Tipo	Descrizione
DISCARD_REASON	String	Motivo dello scarto
REUSE	String	Flag di riutilizzo: S (riutilizzabile) o N (non riutilizzabile)
DISCARD_DATE	Timestamp	Data e ora dello scarto

Table 3.8: Colonne di metadati aggiunte ai record scartati

Tra i motivi di scarto, possiamo trovare:

- **CAST_ERROR**: errore di conversione del tipo di dato.
- **REQUIRED_FIELD_MISSING**: Campo obbligatorio con valore null.
- **DUPLICATE**: Violazione della chiave primaria.
- **FK_VIOLATION**: violazione della chiave esterna.
- **DOMAIN_CONSTRAINT_VIOLATION**: valore fuori dal dominio ammesso.

Riutilizzo degli scarti

In modalità delta, gli scarti del batch precedente vengono automaticamente recuperati nella fase 5 (*Reuse Discards*) della pipeline.

1. **Batch T:** il record non supera la validazione e viene salvato in `oErrors`.
2. **AWS Pipeline:** il percorso `oErrors` viene copiato su `iErrors` per il batch successivo.
3. **Batch T+1:** la fase *Reuse Discards* carica i record da `iErrors` e li reimmette nel flusso; se ora validi, vengono consolidati nell'output finale.

Ad esempio:

Listing 3.4: Esempio di riutilizzo di uno scarto tra batch consecutivi

```
1 Giorno 1:  
2   - MOVIMENTI con FK su CLIENTI inesistente  
3   - Scartato con FK_VIOLATION, REUSE=S  
4  
5 Giorno 2:  
6   - CLIENTI arriva  
7   - MOVIMENTI recuperato da iErrors  
8   - Validato con successo
```

3.7 Ciclo di vita tra batch consecutivi

Come detto, il collegamento tra un batch e il successivo è garantito dallo step FLOW, che copia gli output del giorno T-1 nelle cartelle di input del giorno T.

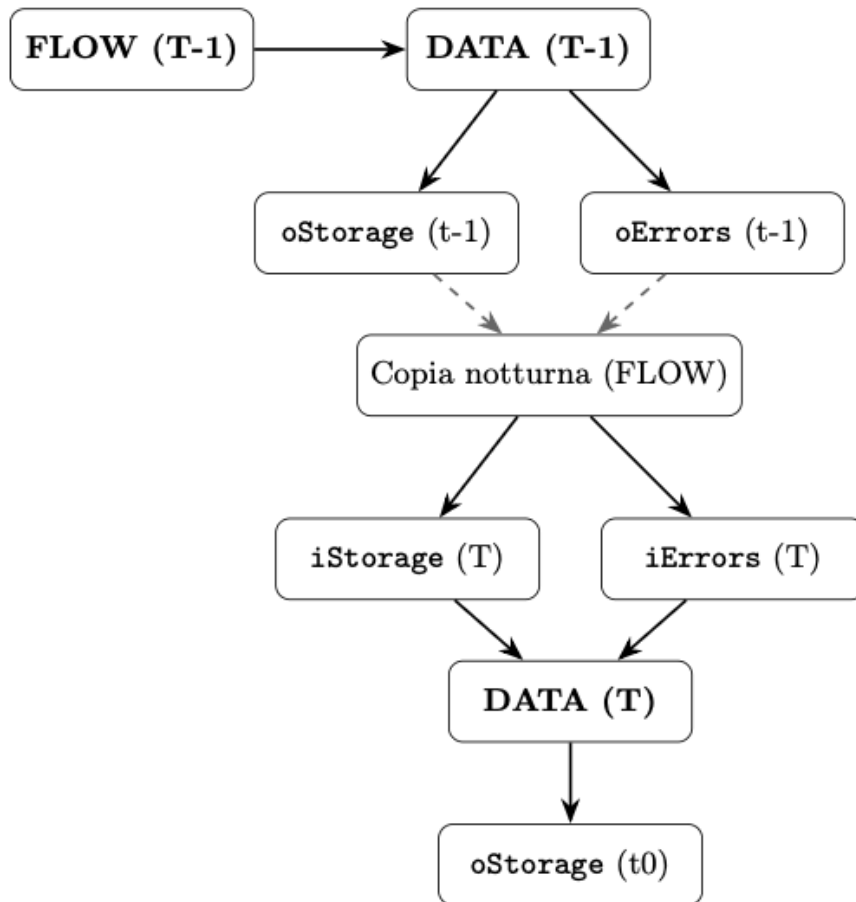


Figure 3.13: Ciclo di vita dei dati tra batch consecutivi.

In sostanza, per ogni tabella in modalità delta:

1. Il batch del giorno T-1 produce oStorage e oErrors.
2. Lo step FLOW del giorno T copia questi output nelle cartelle di input (iStorage e iErrors) per il batch del giorno T.
3. Il batch del giorno T legge lo storico da iStorage, i nuovi dati da source, e gli scarti da iErrors, eseguendo tutte le fasi di validazione e merge.
4. Al termine dell'esecuzione, il batch del giorno T produce un nuovo oStorage (dataset completo aggiornato) e oDeltaStorage (solo le variazioni rispetto allo storico), che saranno a loro volta promossi a input per il batch successivo.

3.8 Flusso completo

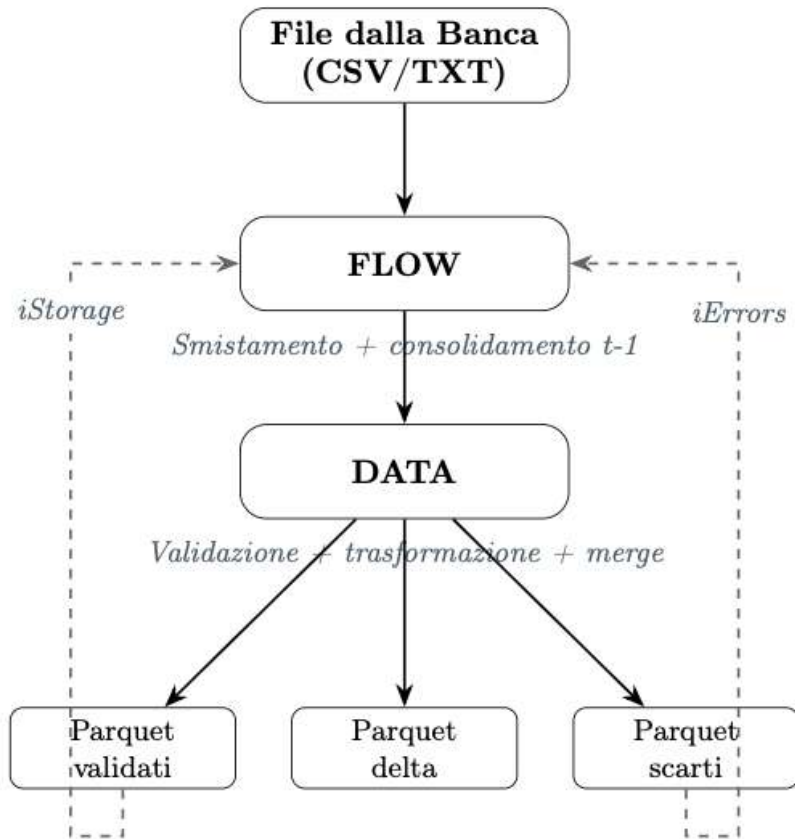


Figure 3.14: Riepilogo del flusso completo di ingestione dati

3.9 Criticità dell'Architettura Corrente

L'architettura attuale, basata su file Parquet grezzi e copie fisiche tra cartelle S3, presenta alcune limitazioni strutturali che compromettono la robustezza, la scalabilità e la manutenibilità della pipeline. Di seguito vengono identificati i principali punti critici emersi durante l'analisi del sistema.

3.9.1 Problema 1: Copia Fisica dei Dati tra Batch Consecutivi

Ad ogni ciclo notturno, lo step FLOW copia fisicamente i file Parquet da `oStorage` a `iStorage` e da `oErrors` a `iErrors`, propagando lo stato del batch precedente come input per quello successivo. Questa operazione introduce le seguenti criticità:

- **Latenza proporzionale al volume:** il tempo di copia scala linearmente con la dimensione dei dataset, allungando la finestra di elaborazione notturna.
- **Duplicazione dello storage:** i dati consolidati coesistono sia in `oStorage` che in `iStorage`, raddoppiando l'occupazione su S3 senza valore aggiuntivo.
- **Fragilità operativa:** una copia fallita o incompleta rende il batch successivo inconsistente, richiedendo intervento manuale per il ripristino.
- **Non idempotenza dello step FLOW:** una riesecuzione accidentale dopo il consolidamento sovrascrive l'input corretto con dati già aggiornati, corrompendo lo stato del sistema.

3.9.2 Problema 2: Assenza di Idempotenza Nativa e di Meccanismi di Rollback

Come discusso precedentemente, l'architettura attuale garantisce una forma di idempotenza condizionale: rieseguire lo stesso batch *prima* del consolidamento non altera l'input, grazie alla separazione tra `iStorage` e `oStorage`. Tuttavia, questa garanzia è fragile per costruzione e non copre tutti gli scenari operativi rilevanti:

- **Assenza di idempotenza post-consolidamento:** se lo step FLOW viene rieseguito per errore dopo aver promosso `oStorage` a `iStorage`, lo stato viene sovrascritto con dati già aggiornati, corrompendo l'input del batch successivo.

- **Mancanza di rollback nativo:** non esiste un meccanismo integrato per ripristinare uno stato precedente; il recupero richiede interventi manuali tramite backup su S3, con rischi di inconsistenza e tempi di ripristino elevati.
- **Complessità dell'orchestrazione difensiva:** per prevenire riesecuzioni accidentali, l'orchestratore (AWS Step Functions) deve implementare logiche di guardia esplicite, aumentando la complessità operativa e il rischio di errori di configurazione.

3.9.3 Problema 3: Aggiornamento Parziale su Record Compositi

Nell'architettura corrente, un record finale può essere il risultato della fusione di più file sorgente eterogenei, ciascuno dei quali contribuisce un sottoinsieme di colonne allo schema complessivo della tabella. Questa struttura introduce inefficienze significative nella gestione degli aggiornamenti incrementali:

- **Ricostruzione integrale del record:** se al giorno T arriva solo un aggiornamento parziale (ad esempio, il solo file dei saldi), il sistema deve comunque ricaricare tutti i file sorgente, ricostruire l'intero record e rieseguire il merge completo, anche per colonne rimaste invariate.
- **Costo computazionale proporzionale al dataset:** il volume di dati processati è determinato dalla dimensione totale dell'output, non dalla quantità effettiva di dati modificati, rendendo l'elaborazione costosa anche in presenza di delta minimi.
- **Assenza di aggiornamento colonnare:** la logica di merge opera a livello di record intero basandosi sulla `primaryKey`; non è possibile aggiornare selettivamente un sottoinsieme di colonne senza rigenerare e riscrivere l'intero record.

3.9.4 Problema 4: Portabilità tra Ambienti

Nell'architettura attuale, i dati sono semplici file Parquet su S3 privi di qualsiasi layer di metadati. Questo rende la migrazione tra ambienti (sviluppo, staging, produzione) banale: è sufficiente copiare i file da un bucket all'altro. Qualsiasi soluzione alternativa deve preservare questa proprietà, che rappresenta un vincolo architetturale rilevante:

- **Metadati con percorsi assoluti:** l'introduzione di un layer di metadati strutturato (manifest file, manifest list, metadata file) implicherebbe la presenza di riferimenti ai percorsi S3 assoluti dei file di dati; una semplice copia tra bucket non sarebbe più sufficiente, poiché i metadati continuerebbero a puntare al bucket di origine.
- **Ambienti su account o region distinti:** gli ambienti di sviluppo, staging e produzione possono risiedere su account AWS o region differenti, ciascuno con il proprio catalogo; la promozione dei dati tra ambienti richiederebbe procedure di migrazione dei metadati non banali.
- **Assenza del problema nell'architettura corrente:** i file Parquet sono autocontenuti e indipendenti dalla loro posizione; non esiste alcun riferimento al percorso fisico all'interno dei dati stessi, rendendo qualsiasi spostamento tra bucket completamente trasparente.

3.9.5 Problema 5: Dipendenza dai Glue Crawler per la Sincronizzazione del Catalogo

Per rendere i file Parquet su S3 interrogabili tramite Amazon Athena o visibili nel Glue Catalog, l'architettura attuale ricorre a Glue Crawler eseguiti come componente separato dalla pipeline. I crawler analizzano la struttura dei file su S3 e creano o aggiornano le definizioni delle tabelle nel catalogo. Questa dipendenza introduce le seguenti criticità:

- **Componente esterno e asincrono:** i Glue Crawler operano al di fuori della pipeline DATA e vengono schedulati separatamente; non esiste garanzia che il catalogo sia allineato con i dati al termine di ogni batch.

- **Riesecuzione manuale a ogni modifica dello schema:** qualsiasi variazione nella struttura dei file Parquet (aggiunta o rimozione di colonne) richiede una riesecuzione esplicita del crawler per propagare le modifiche al catalogo.
- **Rischio di disallineamento:** se il crawler non viene eseguito dopo una modifica, il catalogo continua a esporre lo schema obsoleto, causando errori nelle query Athena o risultati inconsistenti.

3.9.6 Problema 6: Gestione del Compattamento dei File

Nell'architettura corrente, ogni batch riscrive interamente l'output (modalità Full) o aggiunge file delta separati (modalità Delta), ma in entrambi i casi i file scritti sono autocontenuti e indipendenti: non esistono file "obsoleti" da gestire. Qualsiasi soluzione basata su operazioni transazionali di merge o update introduce invece un nuovo onere operativo: ogni scrittura aggiunge nuovi file Parquet marcando quelli precedenti come obsoleti nei metadati, senza rimuoverli fisicamente. Con il susseguirsi dei batch giornalieri, questo comportamento genera le seguenti criticità:

- **Proliferazione di small files:** ogni operazione di merge aggiunge nuovi file senza rimuovere fisicamente quelli sostituiti, causando una frammentazione crescente del dataset su S3.
- **Degrado delle performance di lettura:** un numero elevato di file piccoli aumenta il numero di richieste a S3 per ogni scan, riduce l'efficienza della lettura colonnare e aumenta l'overhead di pianificazione delle query.
- **Crescita incontrollata dello storage:** lo spazio occupato su S3 cresce nel tempo anche in assenza di un aumento del volume logico dei dati, poiché i file obsoleti non vengono eliminati automaticamente.
- **Necessità di un processo di manutenzione dedicato:** a differenza dell'architettura corrente, è necessario introdurre una procedura esplicita di compattamento e pulizia degli snapshot, da orchestrare in modo integrato con la pipeline.

3.10 Vincoli Trasversali

L'analisi delle criticità descritte nelle sezioni precedenti ha evidenziato non solo i limiti dell'architettura corrente, ma anche le proprietà che essa garantisce e che qualsiasi evoluzione deve preservare. Indipendentemente dalle scelte tecnologiche adottate, la nuova soluzione è soggetta a due vincoli fondamentali:

- **Efficienza computazionale:** il costo di elaborazione deve essere proporzionale al volume delle modifiche effettive, non all'intero dataset. Le operazioni di I/O su S3 devono essere minimizzate, evitando riletture o riscritture non necessarie di dati già consolidati.
- **Semplicità di sviluppo:** gli sviluppatori devono poter continuare a lavorare con DataFrame Spark e configurazioni JSON dichiarative. La complessità introdotta da un layer transazionale, quali gestione del catalogo, snapshot o strategie di update, deve essere assorbita dal framework e non esposta al codice applicativo dei singoli moduli.

Chapter 4

Progettazione e Implementazione

4.1 Requisiti progettuali

L'analisi dell'architettura attuale ha fatto emergere come il sistema attuale, pur garantendo affidabilità operativa e soddisfacendo i requisiti funzionali dell'azienda, risulti limitato dall'utilizzo di tabelle Parquet prive di funzionalità transazionali native. Le soluzioni adottate per garantire proprietà quali idempotenza, rollback e gestione dello stato del dato richiedono infatti meccanismi aggiuntivi, introducendo complessità e overhead operativo.

Al fine di modernizzare la piattaforma e superare tali criticità, la progettazione del nuovo sistema è stata guidata dalla necessità di soddisfare i seguenti requisiti fondamentali, direttamente derivati dai problemi precedentemente formalizzati:

- **Eliminazione della duplicazione dello storage:** il batch deve poter leggere lo stato aggiornato del dataset senza che sia necessaria una copia fisica notturna da `oStorage` a `iStorage`.
- **Idempotenza e transazionalità native:** un fallimento durante la scrittura non deve corrompere i dati storici; il rollback a uno snapshot precedente deve avvenire a livello di metadati, senza ripristini manuali.
- **Aggiornamenti granulari:** in presenza di record compositi, un aggiornamento parziale deve modificare solo le colonne coinvolte, senza riscrivere l'intero record.

- **Portabilità dei metadati:** l'introduzione di un livello di metadati non deve ostacolare la migrazione dei dataset tra i diversi ambienti aziendali (sviluppo, staging, produzione).
- **Gestione automatica del catalogo:** la sincronizzazione tra dati e Glue Catalog deve avvenire automaticamente al termine di ogni job, eliminando la dipendenza dai Glue Crawler.
- **Maintenance automatizzata:** compaction dei small file e scadenza degli snapshot obsoleti devono essere orchestrati dalla piattaforma, senza interventi manuali.

Criticità	Requisito progettuale
Duplica storage iStorage/oStorage	Eliminazione della duplicazione dello storage
Step FLOW	Eliminazione del consolidamento
Crawler	Gestione automatica del catalogo
Assenza ACID	Transazionalità nativa
Record compositi	Aggiornamenti granulari
Metadata distribuiti	Portabilità dei metadati
Maintenance manuale	Automazione della maintenance

Table 4.1: Mappatura tra criticità dell'architettura corrente e requisiti progettuali

4.2 Scelta Tecnologica e Architetture

La scelta architetturale centrale consiste nell'introduzione di **Apache Iceberg** come Open Table Format, interposto tra il livello di elaborazione (Apache Spark su AWS Glue) e il livello di storage (Amazon S3). Iceberg aggiunge un layer di metadati versionati sopra i file Parquet esistenti, abilitando transazionalità ACID, snapshot isolation e gestione evoluta dello schema senza modificare il formato fisico dei dati.

Rispetto ad alternative concorrenti come Delta Lake e Apache Hudi, Iceberg è stato preferito per tre ragioni principali: la natura completamente open-source della specifica, l'astrazione pulita del layer dei metadati indipendente dal motore di calcolo, e l'integrazione nativa con l'ecosistema AWS, incluso il Glue Catalog e le Amazon S3 Tables [Ama25a], che adottano Iceberg come formato di riferimento.

L'adozione di Iceberg preserva i due pilastri infrastrutturali esistenti:

- **Amazon S3** continua a ospitare i dati in formato Parquet, affiancati dall'albero dei metadati Iceberg (metadata file, manifest list, manifest file).
- **Apache Spark** (tramite AWS Glue Jobs) rimane il motore di calcolo distribuito per la logica di Data Integration.

Eliminazione dei Glue Crawler

Nell'architettura precedente la visibilità delle tabelle su Amazon Athena era demandata ai Glue Crawler, processi asincroni ed esterni alla pipeline che ricostruivano il catalogo analizzando periodicamente il contenuto di S3.

Iceberg elimina questa dipendenza grazie all'integrazione nativa con AWS Glue Catalog: al termine di ogni operazione di scrittura o MERGE, il framework esegue un commit atomico verso le API del catalogo, aggiornando il puntatore al nuovo `metadata.json`. Se il commit va a buon fine, dati e schema sono immediatamente visibili a tutti i client a valle; se fallisce, il catalogo resta invariato e l'operazione viene interamente annullata.

Il risultato è un catalogo sempre allineato con lo stato effettivo dei dati, senza fasi di orchestrazione aggiuntive né rischi di disallineamento tra schema e storage.

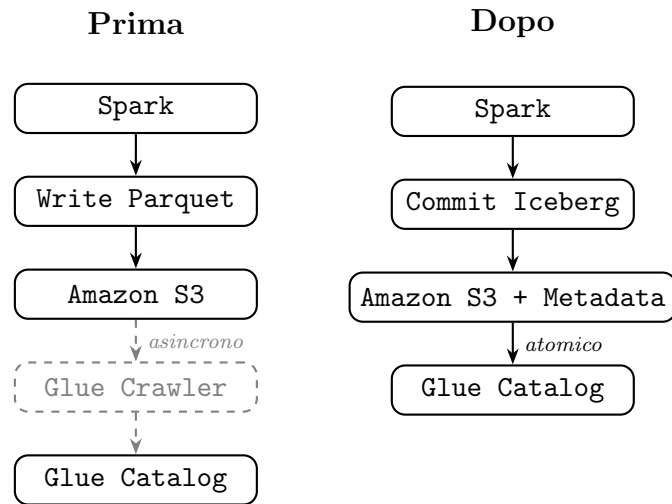


Figure 4.1: Gestione del catalogo: confronto tra architettura precedente (con Glue Crawler) e nuova (con Iceberg)

4.3 Progettazione della nuova pipeline ETL

L'adozione di Apache Iceberg ha reso possibile una revisione profonda del flusso di elaborazione, eliminando alcune delle ridondanze strutturali dell'architettura precedente e consentendo di semplificare sia il ciclo di vita del dato tra batch consecutivi, sia la configurazione dichiarativa delle tabelle, sia la logica applicativa dei job Spark. Le sezioni seguenti descrivono le scelte progettuali e implementative adottate, in risposta diretta alle criticità identificate nei capitoli precedenti.

4.3.1 Superamento del consolidamento fisico: eliminazione dello step FLOW

Il problema centrale dell'architettura precedente era la necessità di copiare fisicamente l'output di ogni batch (`oStorage`) come input del batch successivo (`iStorage`). Questa operazione, orchestrata dallo step FLOW, generava una duplicazione dello storage, necessitando di elaborazioni notturne e rappresentando un punto di fragilità per l'idempotenza dell'intero sistema.

Con Iceberg, questa ridondanza scompare: ogni scrittura o aggiornamento pro-

duce un nuovo snapshot immutabile tramite un commit atomico sui metadati, senza sovrascrivere i dati esistenti. Il batch successivo legge direttamente lo snapshot corrente della tabella, rendendo superflua qualsiasi directory di appoggio o copia fisica.

Di conseguenza, le responsabilità dello step **FLOW** sono state drasticamente ridotte. La gravosa fase di consolidamento e copia notturna è stata interamente eliminata. Nella nuova pipeline, l'orchestrazione del **FLOW** si limita esclusivamente allo smistamento dei file sorgente grezzi in arrivo dalla banca verso le rispettive cartelle **import/**, fungendo da semplice innesco per il modulo **DATA**.

I benefici diretti sono:

- **Eliminazione della duplicazione dello storage:** i dati esistono in un'unica tabella Iceberg su S3; non esiste più la distinzione tra copia di input e copia di output.
- **Rollback senza interventi manuali:** gli snapshot precedenti restano accessibili tramite time travel; ripristinare uno stato precedente è un'operazione a livello di metadati, senza spostamenti fisici di file.
- **Idempotenza strutturale:** ogni commit Iceberg è atomico; una riesecuzione accidentale del batch non corrompe lo stato della tabella, poiché il commit precedente rimane lo snapshot corrente fino al completamento con successo di uno nuovo.

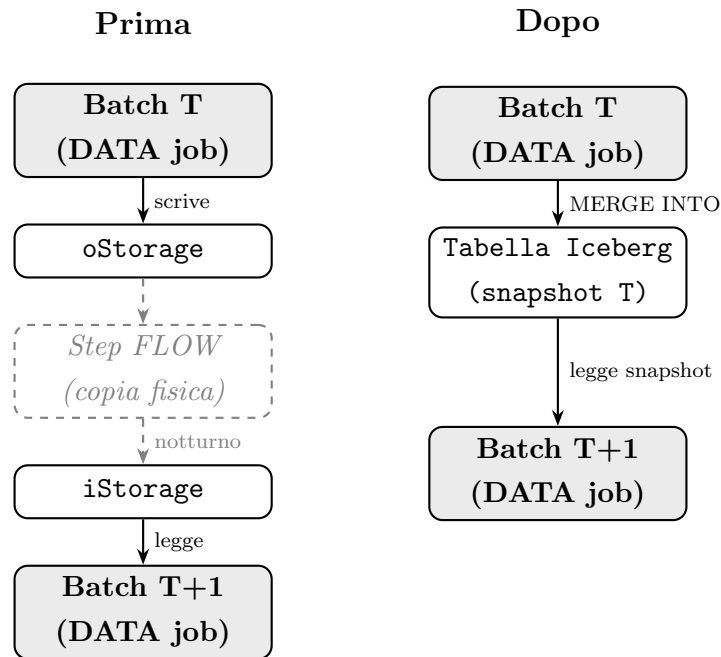


Figure 4.2: Confronto tra architettura precedente (con copia FLOW) e nuova (con Iceberg)

4.3.2 Gestione dello Stato e Idempotenza: il Cursor Watermark

Con Parquet, la replicabilità era garantita strutturalmente dalla separazione tra **iStorage** e **oStorage**: rieseguire il batch del giorno T significava trovare sempre gli stessi file di partenza, perché lo step FLOW aggiornava **iStorage** solo al termine del consolidamento.

Con Iceberg questa separazione scompare: le tabelle sono *live* e puntano sempre allo snapshot più recente. Senza un meccanismo esplicito, rieseguire il batch troverebbe uno stato già modificato dal run precedente.

La soluzione adottata è un **cursor watermark** per-database: al termine di ogni batch completato con successo, il framework scrive un file JSON nella directory `_watermark/` del warehouse Iceberg, registrando il timestamp del completamento e la lista delle tabelle coinvolte.

Listing 4.1: Formato del cursor watermark

```
1 {  
2   "updatedAt": "2026-04-17T14:26:34",  
3   "batchTimestamp": 1776428794742,  
4   "tables": [  
5     "glue_catalog.pfp.MOVIMENTI",  
6     "glue_catalog.pfp.CLIENTI"  
7   ]  
8 }
```

Alla prima esecuzione assoluta il file non esiste ancora: il framework assegna quindi `null` allo storico di ogni tabella e le costruisce da zero. Al termine del batch il cursor viene scritto per la prima volta.

Nelle esecuzioni successive, il cursor viene letto prima di qualsiasi operazione sulle tabelle. Lo storico di ogni tabella viene caricato tramite Spark SQL con la sintassi `SELECT * FROM <tabella> TIMESTAMP AS OF '<batchTimestamp>'` anziché dalla versione live, garantendo che il punto di partenza sia identico a quello del batch precedente. Se il batch fallisce con un'eccezione, il cursor non viene aggiornato: il prossimo tentativo ripartirà automaticamente dallo stato corretto, senza interventi manuali.

A supporto dello sviluppo, il framework espone una **modalità** `--dev-mode`: il cursor non viene aggiornato a fine batch e, prima di ogni riesecuzione, le tabelle vengono automaticamente riportate allo snapshot consolidato di $t-1$ tramite la procedura nativa di Iceberg `rollback_to_timestamp(...)`. Lo sviluppatore può così eseguire il batch quante volte vuole, cambiando parametri, configurazioni o trasformazioni, ripartendo sempre dallo stesso stato di partenza, senza alcun intervento manuale sulle tabelle.

4.3.3 Evoluzione della Configurazione

Ogni tabella del framework è descritta da un file JSON che ne specifica sorgente, schema, chiavi e comportamento di merge. Come mostrato nel Listing 3.2, la configurazione in modalità delta richiedeva cinque percorsi S3 distinti: lo storico di input (`iStorage`, `iErrors`), la destinazione di output (`oStorage`, `oErrors`) e il delta giornaliero (`oDeltaStorage`). Questi percorsi erano espressi come path assoluti, rendendo la configurazione dipendente dall'ambiente di esecuzione.

Con Iceberg, la gestione dello stato storico è delegata al formato stesso: il cursor watermark sa da quale snapshot leggere, e il catalogo Glue conosce la posizione fisica della tabella. I soli campi specifici per Iceberg da aggiungere alla configurazione esistente sono `storageFormat` e `writeMode`; tutto il resto — schema, chiavi, trasformazioni — rimane invariato:

Listing 4.2: Configurazione tabella in modalità Iceberg Delta

```
1 {  
2   "name": "MOVIMENTI",  
3   "source": "import/MOVIMENTI/movimenti.csv",  
4   "storageFormat": "ICEBERG",  
5   "writeMode": "delta",  
6   "primaryKey": ["CODICEBANCA", "CODICE"]  
7   // "columns", "foreignKey", ... rimangono invariati  
8 }
```

Il campo `source` rimane invariato, in quanto descrive la sorgente esterna indipendentemente dal formato di storage. Ciò che scompare sono i cinque percorsi S3 che descrivevano lo stato interno della pipeline: `iStorage`, `oStorage`, `iErrors`, `oErrors` e `oDeltaStorage`. Il campo `storageFormat` attiva il percorso Iceberg all'interno del framework; `writeMode` distingue tra scrittura completa (`full`) e incrementale (`delta`). I percorsi fisici su S3 sono derivati automaticamente dal warehouse Iceberg configurato a livello di ambiente, eliminando la dipendenza da path assoluti nella configurazione della singola tabella.

4.4 Amazon S3 Tables e Automazione della Maintenance

4.5 Migrazione dei metadata

script per spostare dati tra ambienti diversi

4.6 Gestione dei record compositi

- Design della soluzione (To-Be): Come cambia l'architettura logica.
- Configurazione dello Stack: Integrazione di Iceberg con Spark su AWS.
- Ottimizzazione: Implementazione di processi di Maintenance (Compaction dei file, rimozione dei vecchi snapshot/orphan files).

Chapter 5

Test e risultati

- Benchmark di Performance: Confronto tra il vecchio sistema e il nuovo (tempi di esecuzione del batch).
- Affidabilità: Dimostrazione del supporto alle transazioni ACID (es. cosa succede se un job fallisce a metà).
- Costi: Impatto dell'ottimizzazione dei file (Compaction) sui costi di storage.
- Risposta ai 6 punti critici

PARQUET							
	Succeeded	0	05/12/2026 15:33:13	05/12/2026 15:41:30	8 m 11 s	4 DPU/s	G.1X 5.0
ICEBERG							
	Succeeded	0	05/13/2026 12:59:23	05/13/2026 13:07:24	7 m 55 s	4 DPU/s	G.1X 5.0
S3TABLES							
	Succeeded	0	05/13/2026 11:20:09	05/13/2026 11:27:31	7 m 18 s	4 DPU/s	G.1X 5.0

Figure 5.1: Massive test.

Chapter 6

Conclusioni e sviluppi futuri

- Risultati raggiunti: Sintesi dei benefici ottenuti dall'azienda.
- Limiti riscontrati: Eventuali criticità del framework o dei servizi AWS durante lo sviluppo.
- Evoluzioni: aggiornamento parziale sui record compositi

Bibliography

- [Ach24] Alessandro Achilli. Data lakehouse: cos'è, architettura e confronto con data warehouse. <https://www.it-impresa.it/blog/data-lakehouse/>, 2024.
- [AGX⁺21] Michael Armbrust, Ali Ghodsi, Reynold Xin, Matei Zaharia, et al. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of CIDR*, 8(1):28, 2021.
- [Ama25a] Amazon Web Services. Amazon s3 tables documentation. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables.html>, 2025.
- [Ama25b] Amazon Web Services. Aws glue documentation, 2025. <https://docs.aws.amazon.com/glue/>.
- [Apa25a] Apache Software Foundation. Apache iceberg documentation, 2025. <https://iceberg.apache.org>.
- [Apa25b] Apache Software Foundation. Apache iceberg specification. <https://apache.github.io/iceberg/spec/>, 2025.
- [Apa25c] Apache Software Foundation. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2025.
- [Apa25d] Apache Software Foundation. Apache spark documentation. <https://spark.apache.org/docs/latest/>, 2025.
- [BGK21] Edmon Begoli, Ian Goethert, and Kathryn Knight. A lakehouse architecture for the management and analysis of heterogeneous data for

- biomedical research and mega-biobanks. In *2021 IEEE International Conference on Big Data (Big Data)*, pages 4643–4651. IEEE, 2021.
- [BN21] Vladimir Belov and Evgeny Nikulchev. Analysis of big data storage tools for data lakes based on apache hadoop platform. *International Journal of Advanced Computer Science and Applications*, 12(8), 2021.
- [CD97] Surajit Chaudhuri and Umeshwar Dayal. An overview of data warehousing and olap technology. *ACM Sigmod record*, 26(1):65–74, 1997.
- [DB26] Confluent Developer and Tim Berglund. Iceberg + tableflow: Confluent course on apache iceberg, 2026. YouTube playlist, <https://www.youtube.com/playlist?list=PLf38f5LhQthcLN84xP6JgQ-pEbAc8SbSi>.
- [el] exact lab. Data lake in cloud. <https://www.exact-lab.it/en/data-lake-and-iot-simple-safe-efficient/>.
- [Inm95] William H Inmon. What is a data warehouse? *Prism Tech Topic*, 1(1):1–5, 1995.
- [Mac23] Joseph Machado. What is an open table format? why to use one? https://www.startdataengineering.com/post/what_why_table_format/, 2023.
- [Mer23] Alex Merced. Exploring the architecture of apache iceberg, delta lake, and apache hudi. <https://www.dremio.com/blog/exploring-the-architecture-of-apache-iceberg-delta-lake-and-apache-hudi>, 2023.
- [Nix24] Michael Nixon. Data warehouse, data lake e data lakehouse: Tutto quello che c'è da sapere. <https://www.snaplogic.com/it/blog/data-warehouses-data-lakes-data-lakehouses-everything-you-need-to-know>, 2024.
- [NS25] Srinivasa Rao Nelluri and FA Albert Saldanha. Mastering big data formats: Orc, parquet, avro, iceberg, and the strategy of selection.

BIBLIOGRAPHY

- International Journal of Computer Trends and Technology*, 73(1):44–50, 2025.
- [Sah24] Dipankar Saha. Disruptor in data engineering-comprehensive review of apache iceberg. *SSRN Electronic Journal*, 2024.
- [SGL⁺24] Jan Schneider, Christoph Gröger, Arnold Lutsch, Holger Schwarz, and Bernhard Mitschang. The lakehouse: State of the art on concepts and technologies. *SN Computer Science*, 5(5):449, 2024.
- [Sta24] Databricks Staff. What is a data lakehouse? <https://www.databricks.com/blog/what-is-data-lakehouse>, 2024.
- [Sta25] Qlik Staff. What is parquet file format? use cases and benefits. <https://www.qlik.com/blog/what-is-the-parquet-file-format-use-cases-and-benefits>, 2025.

Acknowledgements

Optional. Max 1 page.